

Enhancing the Security of Large Character Set CAPTCHAs Using Transferable Adversarial Examples

Guoheng Sun^{*}, Yucheng Fu^{*}, Han Yang, Juntian Huang, Ruimei Zhang, Haizhou Wang[†], *Member, IEEE*,

Abstract—The large character set CAPTCHA is an important extension of the traditional text-based CAPTCHA with larger alphabet languages to defend against automated attack programs. However, the state-of-the-art deep learning attacks have cracked such CAPTCHA. Existing defenses against such threats increase the complexity of CAPTCHA, thus decreasing usability. We propose ACG (Adversarial Large Character Set CAPTCHA Generation), a framework with two modules: a *Fine-grained Generation Module*, combining three novel strategies to prevent attackers from recognizing characters, and an *Ensemble Generation Module* to generate global perturbations in CAPTCHAs. It not only strengthens defense against recognition attacks but also improves robustness against diverse detection architectures through adversarial perturbations. Additionally, we develop a toolkit, Adv-Eval, consisting of CAPTCHA datasets from 10 of the most popular Chinese CAPTCHA schemes and benchmarking various attacks. We conduct extensive experiments using Adv-Eval to demonstrate ACG's efficacy, especially manifesting a significant decrease in the average success rate of diverse attacks from 51.52% to 2.56%. To the best of our knowledge, ACG is the first framework to defend large character set CAPTCHAs against detection attacks using transferable adversarial examples.

Index Terms—CAPTCHA, deep learning, adversarial example, ACG framework.

I. INTRODUCTION

CAPTCHA, short for Completely Automated Public Turing Test to Distinguish Between Humans and Automated Computer Programs, serves as a crucial tool for distinguishing between human beings and automated scripts. Currently, CAPTCHAs are widely deployed to fight against the excessive use of web crawlers for information gathering [1], serving as a robust password [2], and for authentication in mobile devices [3], etc. To meet specific security and user experience criteria, various classes of CAPTCHAs have been developed: The text-based CAPTCHA [4] asks users to recognize some characters in the CAPTCHA to pass the test; the image-based CAPTCHA [5] requires users to recognize and select specific images in the CAPTCHA; the audio-based CAPTCHA [6] asks users to identify audio information in CAPTCHA; the behavior-based CAPTCHA [7] was recently proposed to integrate action analysis into traditional text-based or image-based CAPTCHA, such as sliding and clicking [8]. Despite the existence of numerous CAPTCHA schemes, text-based schemes continue to dominate because of their cost-effectiveness in implementation and good user experience.

The authors are with the School of Cyber Science and Engineering, Sichuan University, Chengdu, China. *: These authors contributed equally to this work. (Corresponding author [†]: Haizhou Wang.)

A. Large Character Set CAPTCHA

In fact, all the CAPTCHAs mentioned above are vulnerable to attacks by well-designed Deep Neural Networks (DNNs) [4], [5], [8]–[11]. As for conventional text-based CAPTCHAs that only utilize English letters and Arabic numerals, a significant vulnerability arises from their constrained classification space when exposed to DNN-based attacks [12] because it is typically easier for deep learning models to perform small-target classification tasks [13]. To this end, the designers of CAPTCHAs have introduced text-based CAPTCHAs with larger character sets, which ask users to recognize characters from languages such as Chinese [12], [14], Japanese [14], Hindi [15], [16], and Korean [17]. These CAPTCHAs expand the range of classification targets to enhance their resistance to attacks from deep-learning-based models. Current large character set CAPTCHAs can be categorized into the input-based and click-based schemes [12], where the input-based scheme requires users to input characters in the CAPTCHA and the click-based scheme requires users to click the characters in the CAPTCHA.

Although a number of studies have focused on strengthening large character set CAPTCHAs [18]–[20], due to the rapid evolution of deep learning technology, such CAPTCHAs are losing their security at an alarming rate [10], [12], [14], [21]. It is noteworthy that Wang et al. [12] have cracked 11 commonly employed Chinese CAPTCHAs, attaining a notable maximum attack success rate of 86.9%. While some endeavors have aimed to enhance their security [12], [14], [22], they either increase the complexity of CAPTCHAs or use additional verification processes, which decreases usability of CAPTCHAs [23]. Moreover, once attackers collect higher-quality datasets and construct more advanced attack models, these defense measures will still be cracked [4], [5], [12], [24]. Considering the widespread usage of large character set CAPTCHAs, which cater to billions of users [12], it is imperative to present novel defense methods to enhance their security.

B. Defending Attacks Using Adversarial Examples

After carefully analyzing recent attacks on large character set CAPTCHAs, we summarize that they are mainly cracked due to two types of attacks, which are shown in Fig. 1. **(1) Character Detection Attacks [5], [10], [12]:** With advanced object detection models or segmentation techniques, attackers can locate characters in the CAPTCHAs and thus launch

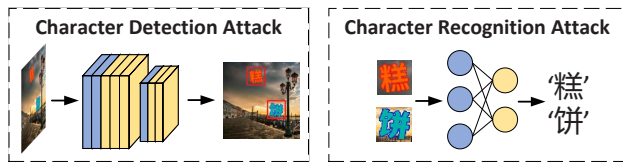


Fig. 1: Examples of Attacks on Large Character Set CAPTCHAs

further attacks. (2) **Character Recognition Attacks** [4], [12], [21]: When all the characters in a CAPTCHA are accessible, attackers can employ character recognition models to recognize them.

Interestingly, recent works have found that by adding imperceptible perturbations to the inputs, one can mislead DNNs [25], [26]. By adding perturbations to generate adversarial examples of large character set CAPTCHAs (i.e., adversarial large character set CAPTCHAs), we can effectively defend against character recognition and detection attacks without significantly affecting usability. When generating adversarial large character set CAPTCHAs, it is challenging to access the models' architectures and parameters on the attacker's side. Specifically, we can only train some **white-box models** to generate adversarial CAPTCHAs and use them to defend against attacks from unknown models, i.e., **black-box models**. Therefore, our primary goal in this paper is to design effective adversarial CAPTCHA generation methods with high transferability (generated by known white-box models, but the defensive capability can transfer to unknown models in the black-box setting).

C. Contributions

In this work, we propose a solution to defend against DNN-based attacks on large character set CAPTCHAs. We summarize our contributions as follows.

- **The first adversarial-example-based framework to defend against black-box attacks on large character set CAPTCHAs.** Compared to existing work using distortion to make large character set CAPTCHAs more complex, our framework injects a restricted magnitude of adversarial perturbations into CAPTCHAs to enhance its security against black-box DNNs, while maintaining its usability.
- **The first defense against character detection attacks on CAPTCHAs using adversarial example techniques.** Concretely, we propose an ensemble method for generating global noise on CAPTCHA images to create adversarial CAPTCHAs, which deviate the predicted character locations from the ground truth locations. This method can also be applied to other CAPTCHAs vulnerable to object detection attacks, such as icon-click-based and sliding-based CAPTCHAs.
- **Implementation of a versatile toolkit (Adv-Eval) to generate and evaluate adversarial large character set CAPTCHAs.** It integrates datasets of popular large character set CAPTCHA schemes, various attack models,

and adversarial CAPTCHA generation methods. **Adv-Eval**¹ can serve as a benchmark for the CAPTCHA security research community, which helps researchers quickly validate their defense designs.

- **Comprehensive Evaluation.** We conducted various evaluations to validate the security, robustness, and usability of our adversarial large character set CAPTCHAs. Importantly, the evaluation results show that our defense method reduces the average success rate of diverse black-box attacks from 51.52% to 2.56%. Moreover, it maintains high defensive capability even when adversarial training and filtering techniques are deployed in the attack. Meanwhile, it maintains good usability for humans according to the results of the user study.

We declare that part of this work has been published in Proceedings of the 33rd International Joint Conference on Neural Networks (IJCNN 2023) [27]. For this journal version, we refine and extend much of our work, specifically described below. (1) **Defensive capability has been further improved by a new global noise generation method.** Concretely, we propose an effective ensemble method that generates global noise across the whole CAPTCHA rather than only in the background area in [27] (Section III-C). As a result, the perturbation against detection attacks on characters covers more features that can mislead the detection models, thus significantly improving the defense ability against character detection attacks. Meanwhile, it achieves certain improvements against recognition attacks because of the further optimization considering recognition models and generalizability from the ensemble method (Section V-B). (2) **The categories of CAPTCHA schemes used for evaluation have been increased.** We collect more CAPTCHA samples from two additional websites, 58.com and ishumei.com, which helps better demonstrate the effectiveness of our method (Section V-B). (3) **Extensive evaluations have been conducted.** To give more convincing results that the adversarial CAPTCHAs we generated are robust to adversarial training, we extend the evaluation of robustness from 4 to 10 CAPTCHA schemes, each with 1,000 samples (Section VI-C1). We also conducted experiments to evaluate the robustness of adversarial CAPTCHAs under various filters (Section VI-C2). We compared ACG with traditional CAPTCHA hardening techniques, demonstrating its advantages (Section VI-D). Moreover, we demonstrate the usability of our adversarial CAPTCHAs by conducting a user study (Section VI-E). (4) **A comprehensive toolkit named Adv-Eval for evaluation has been designed.** It contains a high-quality dataset, an extensive model library, and multiple methods for generating adversarial CAPTCHAs, enabling comprehensive benchmarking of the entire pipeline for both generating and evaluating adversarial CAPTCHAs (Section IV).

D. Technical Overview

At a high level, our proposal is a novel framework to generate large character set CAPTCHAs with adversarial

¹<https://github.com/slghhh/Adversarial-Large-Character-Set-CAPTCHA-Generation>

TABLE I: Summary of Works on Attacking Large Character Set CAPTCHAs

Attack	Segmentation	Recognition Model	Attack Target
Algwil et al. [14]	Preprocessing Techniques	CNN-based Model	Synthetic CAPTCHA
Lin et al. [21]	—	CNN-based Model	Synthetic CAPTCHA
Tang et al. [5]	Preprocessing Techniques	CNN-based Model	Real-world CAPTCHA
Wang et al. [12]	Faster RCNN	Attention-based Model	Real-world CAPTCHA
Zi et al. [4]	—	Attention-based Model	Synthetic CAPTCHA
Nian et al. [10]	Mask-RCNN-based Model	—	Synthetic CAPTCHA

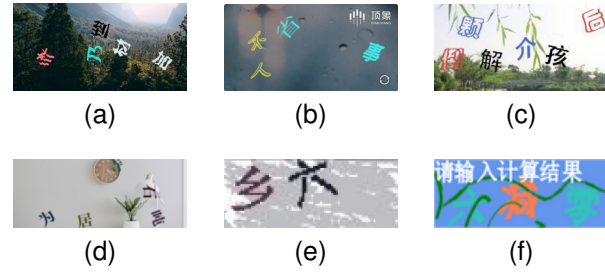


Fig. 2: Examples of large character set CAPTCHAs: (a) Yidun, (b) Dingxiang, (c) YY, (d) Baidu, (e) Renmin, (f) Sougou, where (e) and (f) are input-based while others are click-based

perturbations, namely the Adversarial Large Character Set CAPTCHA Generation framework (ACG framework). ACG has two modules: *Fine-grained Generation Module* for generating partial noise on characters (Section III-B) and *Ensemble Generation Module* for generating global noise on the CAPTCHA image (Section III-C). Firstly, in the *Fine-grained Generation Module*, we feed all the characters in a CAPTCHA and obtain characters with transferable adversarial perturbations, which focuses on **Character Recognition Attacks**. We achieve this by a carefully designed approach integrating three adversarial example generation strategies to construct an algorithm named ATT-VNI-CT-FGSM. These strategies include optimizing the gradients of the white-box model [28], [29], applying input transformation [26], [28], [30], and our improved attention mechanism. Secondly, in the *Ensemble Generation Module*, we start with the CAPTCHA containing perturbed characters and add global perturbation using an ensemble loss function for multi-task learning, which serves for the following optimization to generate perturbations on the whole CAPTCHA image. As a result, we achieve solid defenses on both black-box **Character Recognition Attacks** and **Character Detection Attacks**.

II. RELATED WORK

A. Attacks on Text-based CAPTCHAs

One basic CAPTCHA scheme deployed by a large number of websites [10] is text-based CAPTCHA. However, these CAPTCHAs have been proven vulnerable to some advanced attacks [4], [9], [10], [31]–[33]. Gao et al. [33] used Convolutional Neural Network (CNN) models to crack the two-layer Microsoft CAPTCHAs with a success rate of 44.6%. As deep learning techniques keep advancing, Zi et al. [4] employed a generic end-to-end attack using CNN and an attention-based Recurrent Neural Network, which cracked most defense mechanisms with higher success rates and faster speeds. Furthermore, Li et al. [9] deployed text-based CAPTCHA generators using Cycle-GAN to expand the datasets for training attack models. They also cracked 10 popular text-based CAPTCHAs with success rates between 22.4% and 88.4%. Nian et al. [10] introduced another end-to-end strategy, which uses only advanced object detectors to locate and recognize characters.

B. Countermeasures on Large Character Set CAPTCHAs

Different from the traditional defense mechanisms, including Overlapping, Hollow, Multi-Font, etc., [4], the large character set, as another popular defense strategy for text-based CAPTCHAs is proposed. We give some examples in Fig. 2.

However, large character set CAPTCHAs are also vulnerable to various attacks [4], [5], [10], [12], [14], [21]. We provide a brief summary in Table I. Taking Chinese CAPTCHAs as examples, Algwil et al. [14] proposed to recognize Chinese characters in the CAPTCHAs with DNN models. Subsequently, Lin et al. [21] utilized methods based on CNNs to recognize CAPTCHAs in Chinese, which obtain an accuracy of 95.0% for individual characters. Then, a deep learning technique was proposed by Tang et al. [5] to attack real-world Chinese CAPTCHAs. Zi et al. [4] conducted an attack with the end-to-end approach on synthetic Chinese CAPTCHAs. Wang et al. [12] cracked 11 Chinese CAPTCHAs from popular websites, obtaining success rates between 34.7% and 86.9%. They showed that large character set CAPTCHAs on popular websites are not secure anymore. Nian et al. [10] also combined detection and recognition attacks in an end-to-end model to crack Chinese CAPTCHAs. In the evaluation, we also use two state-of-the-art attacks above [10], [12] to test the defensive capability of the proposed method.

In response to the attacks above, researchers have proposed some defense methods to enhance the security of large character set CAPTCHAs. Specifically, Algwil et al. [14] advised enhancing security with segmentation resistance. Unfortunately, there have been two methods [10], [12] designed to crack such a segmentation-resistant CAPTCHA scheme. Furthermore, [12] designed 3D Chinese reasoning CAPTCHAs. However, this scheme was soon cracked by advanced DNN-based attacks [24], [42]. We note that the above two defense strategies are both based on posing extra recognition difficulties, even for human beings. They do not have black-box transferability to newly trained models or robustness against new preprocessing techniques [10], [12], [24], [42]. In other words, attackers can crack such CAPTCHAs by training new models with unknown parameters or by using segmentation techniques that do not rely on DNN models. To the best of our knowledge, only [41] explored the application of adversarial examples on large character set CAPTCHAs. This method is based on the basic FGSM algorithm, without considering more advanced tech-

TABLE II: Comparison with Other Adversarial CAPTCHAs

Method	CAPTCHA Scheme	Defending Attacks	Generation Method
IAN [34]	Image-based CAPTCHAs	Image recognition attacks	IAN-Generation [34]
aCAPTCHA [35]	Text-based / Image-based CAPTCHAs	Text / Image recognition attacks	JSMA ^f / JSMA ⁱ [35]
advCAPTCHA [18]	Text-based CAPTCHAs	Text recognition attacks	Mixture method [18]
RTC [36]	Text-based CAPTCHAs	Text recognition attacks	NI-CT-FGSM [28], [30]
Zhang et al. [37]	Text-based CAPTCHAs	Text recognition attacks	Four module generation [37]
aaCAPTCHA [38]	Audio-based CAPTCHAs	Automatic speech recognition attacks	PGD [39]
GAN-based advCAPTCHA [6]	Audio-based CAPTCHAs	Automatic speech recognition attacks	WaveGAN [40]
Zhang et al. [41]	Click-based Chinese CAPTCHAs	Character recognition attacks	FGSM [25]
ACG (Ours)	Large character set CAPTCHAs	Character recognition attacks, Character detection attacks	ATT-VNI-CT-FGSM, GLOBAL-ENS

niques of adversarial examples. Although it can achieve a high defense success rate against white-box recognition attacks, its transferability to black-box attacks is limited (demonstrated by our evaluation of FGSM in Section VI-A). Moreover, they did not consider defense against character detection attacks, and the robustness of methods against adversarial training is limited, due to the vulnerability of its underlying FGSM algorithm [43].

C. Adversarial Example Techniques

In recent years, DNNs have shown great success in computer vision. However, some researchers discovered that DNNs are vulnerable to input images with well-designed imperceptible perturbations, i.e., adversarial examples [25], [44].

There have been various adversarial example generation methods. In the white-box setting, which assumes that all the parameters and structures of victim models are known, attackers can achieve high attack success rates. However, in practice, we usually have no access to the information of victim models, i.e., a black-box setting [45]. In the black-box scenario, a useful method is taking advantage of the transferability of adversarial examples [6], [26], [28]–[30], [39], [45]–[48]. Kurakin et al. [48] and Madry et al. [39] introduce iterative versions of the basic Fast Gradient Sign Method (FGSM) [25] to improve the transferability. Furthermore, Dong et al. [45] integrate momentum to generate more transferable adversarial examples. Lin et al. [28] use an iterative FGSM based on Nesterov accelerated gradient method to avoid local optima. Wang et al. [29] propose a variance tuning method to achieve stable gradient update to improve the transferability of adversarial examples further. Instead of optimizing the gradient calculation, Xie et al. [26] use a Diverse Input Method to transform input to generate transferable adversarial examples. Lin et al. [28] propose to use a scale-invariant method to expand input images to generate adversarial examples. Dong et al. [30] propose the Translation Invariant Method using translation augmentation to generate transferable adversarial examples. Moreover, Wu et al. [46] propose an attention-based method using class activation maps (CAM) to improve the transferability of adversarial examples. Li et al. [47] also use CAM to construct attention mechanisms to perform a sparse adversarial attack.

Recently, adversarial examples have been designed for object detection models, which can mislead object detectors to give completely wrong location and class predictions of input

images [49]–[51] even in the physical world [52]–[54]. Although adversarial examples can mislead object detectors with various backbones, it is harder for them to have transferability among different architectures.

D. Design CAPTCHAs with Adversarial Example Techniques

Existing research results suggest that adversarial example techniques are helpful for the design of more secure CAPTCHA schemes [6], [18], [34], [35], [37], [41]. We present a short comparison of these methods and ours in Table II. By adding imperceptible noise, we can enhance the security of CAPTCHAs while still ensuring user experience. Osadchy et al. [34] constructed immutable adversarial examples of CAPTCHA images, which are robust against image filters. To secure the text-based CAPTCHAs, Zhang et al. [37] proposed an approach considering four modules. Then, Shi et al. [35] designed adversarial CAPTCHAs with text and images, obtaining high transferability and usability using JSMA [55]. Shi et al. [18] also investigated the real-world deployment of text-based adversarial CAPTCHAs. Shao et al. [36] designed robust text-based adversarial CAPTCHAs with an advanced adversarial example generation method. Wang et al. [6] proposed to use WaveGAN, a GAN-based method to generate audio adversarial CAPTCHAs, which is friendly for people with visual impairment. As for large character set CAPTCHAs, Zhang et al. [41] investigated the defense for Chinese CAPTCHAs with adversarial perturbations using FGSM [25]. Their method focuses on defending basic white-box recognition models.

Different from other works, our method is innovative since it is the first design to generate adversarial examples with good transferability for large character set CAPTCHAs. Therefore, we can defend against the attacks from black-box models. Furthermore, for the first time, we leverage adversarial example techniques to fight against character detection in CAPTCHAs.

III. ACG FRAMEWORK

A. Problem Definition and Overview of the ACG Framework

1) *Framework Overview*: We present the workflow of the ACG Framework in Fig. 3. It can generate adversarial CAPTCHAs with good transferability, robustness, and usability. The ACG framework mainly consists of two modules: *Fine-grained Generation Module*, which generates refined perturbations on characters first, and *Ensemble Generation Module*, which generates global noise in the CAPTCHA with an ensemble loss function.

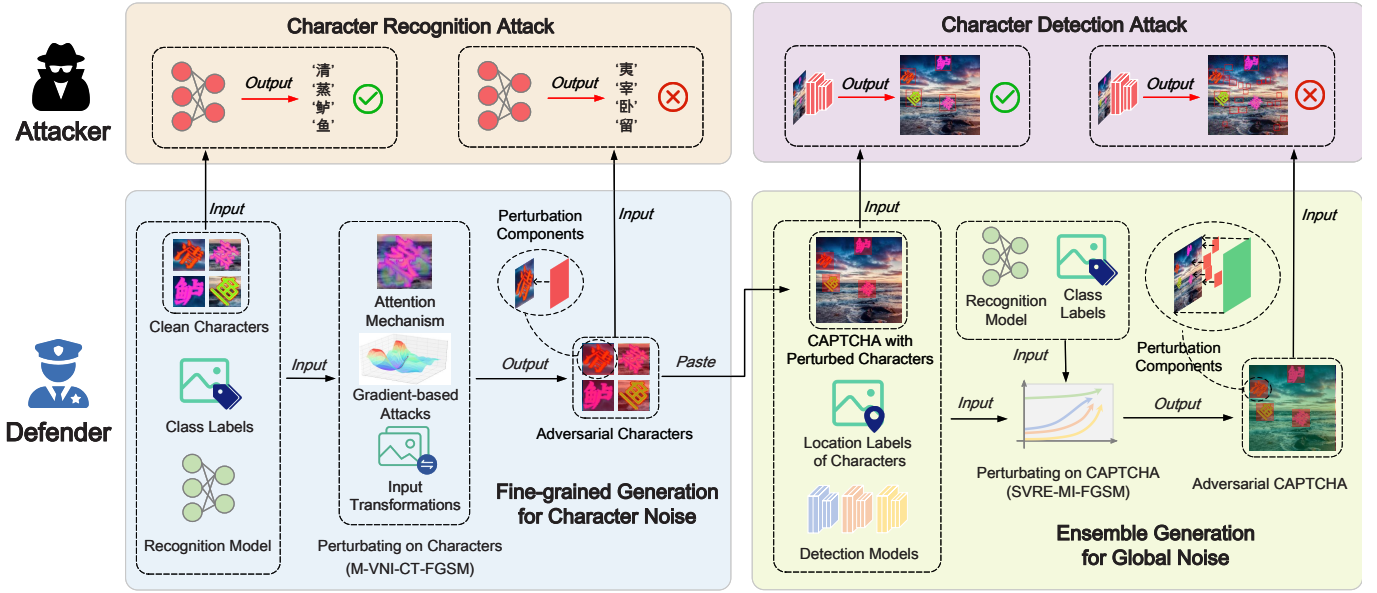


Fig. 3: Overview of the ACG Framework. To aid reader's comprehension, adversarial examples are represented with visually distinguishable colors, and the actual adversarial examples do not appear in this way.

2) *Problem Definition:* We denote a CAPTCHA image by X and a single character in X by x . The output of the ACG framework is a CAPTCHA image with limited perturbation magnitude at the pixel level, denoted by X^{adv} . Concretely, the perturbation magnitude is bounded by the standard L_∞ norm method [18], [46], [50] as $\|X - X^{adv}\|_\infty \leq \epsilon$. That is, for any pixel i , we have $\frac{|X[i] - X^{adv}[i]|}{X[i]} \leq \epsilon$. With the above perturbation bound, the ACG framework aims to achieve the following two forms of defense for the generated adversarial CAPTCHA:

- **Defense against Character Recognition Attacks:** Given a CAPTCHA image X , we crop the characters x by the location labels B . Then we use the category label y , along with a white-box character recognition model f to generate the adversarial character x^{adv} by maximizing the model's loss function $J(x^{adv}, y; \theta)$. Given a recognition model f' (white-box or black-box), x^{adv} misleads the recognition results: $f'(x^{adv}; \theta) \neq y$, i.e. the classification result $f'(x^{adv}; \theta)$ fails to match its labels y .
- **Defense against Character Detection Attacks:** For each CAPTCHA image X with character location label B , we choose white-box detection models $f_0, \dots, f_{n-1}$, which generate adversarial examples X^{adv} after maximizing the models' loss functions $J(X^{adv}, B; \theta_0, \dots, \theta_{n-1})$. As a result, given a detection model f' (white-box or black-box), X^{adv} misleads the detection result: minimize $f'(X^{adv}; \theta) \cap B$, i.e., minimizing the overlap space of predicted character locations $f'(X^{adv}; \theta)$ and the ground-truth character locations B .

In Table III, we summarize the notation used frequently in this paper. Other notations are explained when specific techniques are introduced.

TABLE III: Technical Terms and Notations Used in the Paper.

Notation	Meaning
X	CAPTCHA image
x	Character image
ϵ	Perturbation magnitude
θ	Model parameter
y	Label of recognition task
B	Location label of detection task
$f(x; \theta)$	Output of model θ given input x
$J(x, y; \theta)$	Loss function of recognition model
$J(x, B; \theta)$	Loss function of detection model
$\nabla_x J$	Gradient of J with respect to x

B. Fine-grained Generation Module for Character Noise

This section focuses on **Defending against Character Recognition Attacks**. Since recognition is the key to cracking large character set CAPTCHAs [5], [12], [24], we propose a *Fine-grained Generation* method to first generate adversarial noise on characters. It uses a refined combination of multiple strategies to limit the success rate of recognition attacks on characters. In the next section, we will use *Ensemble Generation* to further optimize the perturbation of characters. Specifically, the *Fine-grained Generation* is based on the classical FGSM algorithm [25] and further improved with advanced adversarial example generation strategies to build ATT-VNI-CT-FGSM in Algorithm 1 for adversarial character generation. Intuitively, our ATT-VNI-CT-FGSM algorithm works as follows. It takes a character image x , its label y , and model parameters θ as inputs and iterates for T rounds to output the adversarial character x_T^{adv} . In each round, it makes data augmentation to x , computes the loss function $J(x_t, y; \theta)$, and derives the gradients g_t . By doing so, we get the gradient information that maximizes the loss function on the input x_t . Finally, the gradients are further refined and added to the current copy of adversarial character x_t . Note that in each

Algorithm 1 ATT-VNI-CT-FGSM

Require: Clean character image x and label y
Require: Character classifier f with parameters θ , category cross-entropy loss function J , maximum perturbations ϵ , additional perturbations within each iteration γ , number of iterations T , momentum decay factor μ , calculation, translation invariance kernel W , number of scaling invariance samples m , diverse input probability p
Ensure: An adversarial example of character image x_T^{adv}

```

1:  $a = \epsilon/T$   $g_0 = 0$ ;  $v_0 = 0$ ;  $x_0^{adv} = x$ 
2: /* Attention Mechanism */
3: Derive  $mask$  by (5)
4: for  $t = 0$  to  $T - 1$  do
5:   /* Gradient-based Attacks */
6:   Nesterov Iteration:  $x_t^{nes} = x_t^{adv} + a \cdot \mu \cdot g_t$ 
7:   Gradient Variance Tuning: Update  $v_{t+1}$  by (3)
8:   /* Input Transformations */
9:   SIM: Scale  $x_t^{nes}$  to a set of images  $x_t^s = S(x_t^{nes}; m)$ 
10:  DIM: Random transform  $x_t^d = D(x_t^s; p)$ 
11:  TIM: Convolution  $x_t^T = W * (\nabla_{x_t^d} J(x_t^d, y; \theta) + v_t)$ 
12:  /* Update Perturbation */
13:  Update gradient  $g_{t+1} = \mu \cdot g_t + \frac{x_t^T}{\|x_t^T\|_1}$ 
14:  Update  $x_{t+1}^{adv}$  with gradient sign and  $mask$ , then clip  $x_{t+1}^{adv}$  by  $\epsilon$ :  $x_{t+1}^{adv} = \text{Clip}_{x, \epsilon}(x_t^{adv} + mask \cdot \text{sign}(g_{t+1}))$ 
15: end for
16: return  $x_T^{adv}$ 

```

update, we apply the operator $\text{Clip}_{x, \epsilon}(\cdot)$, which projects the perturbed character back into the ℓ_∞ -ball around the clean input x , thereby guaranteeing

$$\|x_T^{adv} - x\|_\infty \leq \epsilon, \quad (1)$$

so that every pixel perturbation remains bounded by ϵ . Next, we will elaborate on Algorithm 1 by referring to its three main components.

Gradient-based Attacks (lines 6-7). We consider two types of Gradient-based Attacks, i.e., *Nesterov Iteration* [28] and *Gradient Variance Tuning* [29], to search for better gradients, namely VNI-FGSM. After initializing the gradient $g_0 = 0$ and gradient variance $v_0 = 0$, we gradually add noise to x_t^{adv} . Consider μ as the decay factor for momentum and α as the step size for updating perturbation. For each epoch, x_t^{adv} will first be updated by $x_t^{nes} = x_t^{adv} + \alpha \cdot \mu \cdot g_t$ [28]. Next, we sum up the gradient $\nabla_{x_t^{nes}} J(x_t^{nes}, y; \theta)$ and the gradient variance v_t obtained in the previous epoch [29]. The updated g_{t+1} is obtained by adopting momentum [45] as

$$g_{t+1} = \mu \cdot g_t + \frac{\nabla_{x_t^{nes}} J(x_t^{nes}, y; \theta) + v_t}{\|\nabla_{x_t^{nes}} J(x_t^{nes}, y; \theta) + v_t\|_1}, \quad (2)$$

and the gradient variance v_{t+1} in the next step is derived by

$$v_{t+1} = \frac{1}{N} \sum_{i=1}^N \nabla_{x_{t,i}^{adv}} J(x_{t,i}^{adv}, y; \theta) - J', \quad (3)$$

where $J' = \nabla_x J(x, y; \theta)$, N is the number of neighbor points of $x_{t,i}^{adv}$, $x_{t,i}^{adv} = x_t^{adv} + r_i$ is the i -th sampled point, and

$r_i \sim U[-\beta \cdot \epsilon, \beta \cdot \epsilon]$ consists of values of uniform distribution and has the same shape as x_t^{adv} , β is the factor controlling the sampling range.

Input Transformations (lines 9-13). In the generation of adversarial characters, we employ Composite Transformation (CT), a process that combines three Input Transformations.

- 1) *Diverse Input Method (DIM)* [26]: The image x will be randomly resized and padded with a probability p to alleviate overfitting before being fed to the gradient calculation, which is denoted as $D(x; p)$;
- 2) *Translation-Invariant Method (TIM)* [30]: This approach involves employing a kernel matrix W to perform convolution with the gradient g , represented as $T(g) = W * g$. This achieves translation-based data augmentation.
- 3) *Scale-Invariant Method (SIM)* [28]: This method performs Scale-Invariant transformation to the input x to increase the diversity. Thus, x is expanded to m scaled copies for further gradient calculation. We denote this process as $S(x; m) = \{x/2^0, x/2^1, \dots, x/2^m\}$.

Attention Mechanism (line 3). We use the model's attention to construct a matrix $mask$ (M), which determines the perturbation magnitude for pixels. Specifically, observations in [30], [46] indicate that different models have similar Class Activation Maps (CAM) [56], [57] for correctly recognized images. Encouraged by this observation, we posit that the models employed by attackers similarly encompass common features pertaining to the ground truth y . Therefore, we propose a simple but effective attention mechanism based on Gradient-weighted Class Activation Mapping (Grad-CAM) [56] to perturb characters. First, we derive the spatially pooled gradients by

$$a_k^c[y] = \frac{1}{Z} \sum_m \sum_n \frac{\partial f(x)[y]}{\partial A_k^c[m, n]}, \quad (4)$$

where A_k^c is the c -th feature map in the k -th layer, Z is a normalizing factor for $a_k^c \in [-1, 1]$, $a_k^c[y]$ is the attention weight of the c -th feature map in the k -th convolution layer for class y . Then, rather than using the ReLU function to remove the negative points [56], we derive the absolute value of the weighted sum $\sum a_k^c[y] \cdot A_k^c$ to keep values in the fields with negative contributions to the correct prediction since both positive and negative points in $\sum a_k^c[y] \cdot A_k^c$ can make a difference to the prediction probability of ground truth class y . Then, $\sum a_k^c[y] \cdot A_k^c$ is normalized to $[0, 1]$ and resized to the same shape as the input x by $\hat{H}_k^t = \text{Resize}(\text{Norm}(|\sum_c a_k^c[y] \cdot A_k^c|))$. The resized feature maps can represent the attention weights in each area by the translation-invariance property of CNN. We then construct a $mask$ (M) based on $\hat{H}_k^t$ and α by

$$mask = \alpha + \hat{H}_k^t \cdot \gamma, \quad (5)$$

where $\alpha = \epsilon/T$, ϵ is the maximum magnitude, T is the number of epochs, and γ is the parameter that enlarges the step size for each epoch. It is a matrix specifying the magnitude of perturbation for each pixel, and it can be updated in each epoch during optimization, which makes pixels with more attention weights update with larger step sizes but still clip x_t^{adv} to bound it with $\|x - x_t^{adv}\|_\infty < \epsilon$.

C. Ensemble Generation Module for Global Noise

In this section, we consider both **Defending against Character Detection Attacks** and **Defending against Character Recognition Attacks**. Specifically, after generating noise in characters, we perform an additional step to generate global noise in both the background and character parts, which is distinct from [27], which only adds noise to the background. There are two reasons for this modification. (1) **Eliminating Boundary**. The two steps of noise generation in [27] are independent, causing the obvious boundary between perturbed characters and perturbed backgrounds, which becomes an additional feature for attackers to locate the characters. Our method addresses this issue by directly updating the noise across the whole CAPTCHA to eliminate this boundary. (2) **Improvement in Defensive Capability**. The method in [27] essentially avoids noise generation on characters when defending against detection attacks. However, adding noise to the objects is crucial to generating adversarial examples for object detection [50]. Using the ensemble method that can also perturb the characters when defending against detection attacks, we significantly reduce the detection success rates of detection models and show better defense ability than the method in [27], which can be seen in experiments in Section V-B.

To achieve ensemble generation, we propose a novel loss function for multi-task learning to integrate the loss of recognition and detection for adversarial example optimization.

1) *Loss for Detection Model Architectures*: In practice, attackers' detection models may have different architectures. Therefore, generating adversarial examples that can transfer across various architectures poses a challenge. We integrate the loss functions from different detection model architectures to address this issue. In evaluations, we use FRCNN (Res-101), SSD (Mb-v2), and YOLO-v5 which have different and popular architectures [49] as white-box models.

2) *Loss for the Recognition Model*: As for the recognition model, we follow the white-box recognition model's loss used in the *Fine-grained Generation* step, which does not require additional training for the recognition model. It only takes the characters as input for optimization.

3) *Optimization for Adversarial CAPTCHA Generation*: Given the loss functions, we optimize their weighted sum as (6) to generate adversarial perturbations.

$$J(X, y, B; \theta, w) = \sum_{i=0}^{n-1} w_i J_i(X, B; \theta_i) + w' J'(x, y, \theta'), \quad (6)$$

where J_i is the loss function of the detection model with parameters θ_i and weight w_i , J' is the loss function of recognition model f' with parameter θ' , X is the input CAPTCHA, x denotes all characters in X , B denotes character location labels, y is the character category labels. With the loss function in (6), we use the optimization method SVRE-MI-FGSM [58] based on stochastic variance reduction to generate the final CAPTCHA.

4) *Ensemble Generation Algorithm*: We initialize $X_0^{adv} = X_0$ and step size $a = \epsilon/T$. At the start of each outer iteration t , we evaluate the ensemble loss J_{ens} as in Eq. (6) and

Algorithm 2 GLOBAL-ENS(SVRE)

Require: CAPTCHA X with perturbed characters, clean CAPTCHA X_0 , character categories label y and locations label B , white-box detection models with parameters $\theta_1, \dots, \theta_n$, loss functions $J_1, \dots, J_n$, and their weights $w_1, \dots, w_n$, White-box recognition model with parameter θ' , loss function J' and weight w' , maximum perturbations ϵ , number of inner iterations M , number of external iterations T , iteration momentum decay factor μ_1, μ_2 .

Ensure: An adversarial example of CAPTCHA image X_T^{adv}

```

1:  $a = \epsilon/T$ ;  $g_0 = 0$ ;  $X_0^{adv} = X$ 
2: for  $t = 0$  to  $T - 1$  do
3:   /* Outer Loop Ensemble Loss */
4:   Calculate the loss  $J_{ens}(X_t^{adv}, y, B; \theta, w)$  by (6).
5:   Derive ensemble gradient:  $g_t^{ens} = \nabla_{X_t^{adv}} J_{ens}$ 
6:    $\hat{X}_0 = X_t^{adv}$ ;  $\hat{g}_0 = 0$ 
7:   for  $m = 0$  to  $M - 1$  do
8:     Randomly choose model  $\hat{J} \in \{J_0, \dots, J_{n-1}, J'\}$ 
9:     if  $J'$  is chosen then
10:      /* Derive Gradients in Character Area */
11:      Crop characters  $\hat{x}_m, x_t^{adv}$  from  $\hat{X}_m, X_t^{adv}$  by  $B$ .
12:       $g = \nabla_{\hat{x}_m} \hat{J}(\hat{x}_m, y; \hat{\theta}) - \nabla_x \hat{w} \hat{J}(x_t^{adv}, y; \hat{\theta}) + g_t^{ens}$ 
13:      Pad  $g$  with 0 to the same size as  $X$ .
14:     else
15:      /* Derive Gradients in Global Area */
16:       $g = \nabla_{\hat{X}_m} \hat{J}(\hat{X}_m, b; \hat{\theta}) - \nabla_X \hat{w} \hat{J}(X_t^{adv}, B; \hat{\theta}) + g_t^{ens}$ 
17:     end if
18:      $\hat{g}_{m+1} = \mu_2 \cdot \hat{g}_m + g$ 
19:      $\hat{X}_{m+1} = \text{Clip}_{X_0, \epsilon}(\hat{x}_m + a \cdot \text{sign}(\hat{g}_{m+1}))$ 
20:   end for
21:    $g_{t+1} = \mu_1 \cdot g_t + \hat{g}_M$ 
22:    $X_{t+1}^{adv} = \text{Clip}_{X_0, \epsilon}(X_t^{adv} + a \cdot \text{sign}(g_{t+1}))$ 
23: end for
24: return  $X_T^{adv}$ 
```

take its image gradient $g_t^{ens} = \nabla_{X_t^{adv}} J_{ens}(X_t^{adv}, y, B; \theta, w)$ to provide a global direction that reflects both detection and recognition models. Then we perform M stochastic variance-reduction inner steps by uniformly sampling one model $\hat{J} \in \{J_1, \dots, J_n, J'\}$ each time and forming a control-variate gradient

$$g_m = \nabla_{\hat{Z}_m} \hat{J}(\hat{Z}_m; \hat{\theta}) - \hat{w} \nabla_Z \hat{J}(X_t^{adv}; \hat{\theta}) + g_t^{ens}, \quad (7)$$

where Z is the full image and $\hat{Z}_m$ is either the full image or the cropped character regions (padded back to image size), depending on whether $\hat{J}$ is a detection model or the recognition model. We maintain inner momentum $\hat{g}_{m+1} = \mu_2 \hat{g}_m + g_m$ and, after M steps, aggregate it into the outer momentum $g_{t+1} = \mu_1 g_t + \hat{g}_M$, followed by the ℓ_∞ -projected update

$$X_{t+1}^{adv} = \text{Clip}_{X_0, \epsilon}(X_t^{adv} + a \cdot \text{sign}(g_{t+1})). \quad (8)$$

Using crops allows targeted perturbations on characters, while g_t^{ens} keeps background updates coherent; the dual momenta (μ_1, μ_2) smooth accumulation across inner/outer levels. The projection enforces the noise budget $\|X_T^{adv} - X_0\|_\infty \leq \epsilon$, such that the per-pixel perturbation never exceeds ϵ .

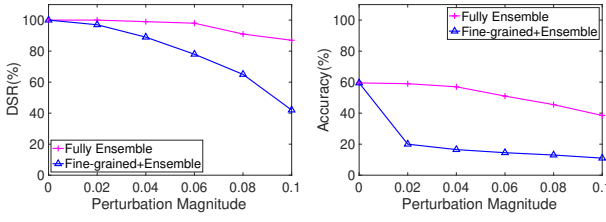


Fig. 4: Attack results of fully ensemble generation method and fine-grained generation + ensemble generation method, measured by detection success rate and recognition accuracy.

The ensemble generation of global noise can also serve as a naive solution, which directly adds noise to the whole CAPTCHA image, considering both character and background. However, the full ensemble method cannot integrate the complex strategies for adversarial characters in *Fine-grained Generation Module*. Even if we manage to integrate fine-grained methods into ensemble methods, it will become very slow. We demonstrate from the empirical results that keeping *Fine-grained Generation* can significantly improve the defensive capability of adversarial CAPTCHA. Fig. 4 compares the defense results directly using the method in this section and firstly using fine-grained generation on characters to generate noise. It shows the character recognition accuracy and detection success rate. Since in the direct ensemble method, noise in characters cannot implement the complex fine-grained methods and also interferes the perturbation against detection, as we can see, the *Fine-grained Generation + Ensemble Generation* shows better defensive capability than the fully *Ensemble Generation* method.

IV. ADV-EVAL TOOLKIT

Overview. As shown in Fig. 5, Adv-Eval is a comprehensive evaluation toolkit with **Datasets** of real-world Chinese CAPTCHAs and synthesized CAPTCHAs as well as a **Model Library** including multiple character detection and recognition models. Users can select CAPTCHA images from the datasets and white-box models for adversarial CAPTCHA generation. These CAPTCHAs can be fed into the black-box models for **Evaluation**.

Datasets. Because of the absence of extensive datasets for real-world large character set CAPTCHAs, we developed several web crawlers, which we used to obtain CAPTCHAs from 10 popular Chinese websites: Yidun (dun.163.com), Dajie (dajie.com), YY (yy.com), Baidu (pan.baidu.com), Dingxiang (dingxiang-inc.com), Shumei (ishumei.com), 58 (58.com), Renmin (people.com.cn), Sougou (sougou.com), Geetest (geetest.com). We categorize CAPTCHAs into input-based and click-based types, with Sougou, and Renmin being the former and others falling into the latter category. Yidun, Dingxiang, Shumei, and Geetest are from CAPTCHA service providers, and others are from popular websites. The collection of CAPTCHAs was performed between June and December 2022. We labeled 2,000 CAPTCHAs for each scheme and used half of them to train models while leaving the other half as the test set for model evaluation and adversarial

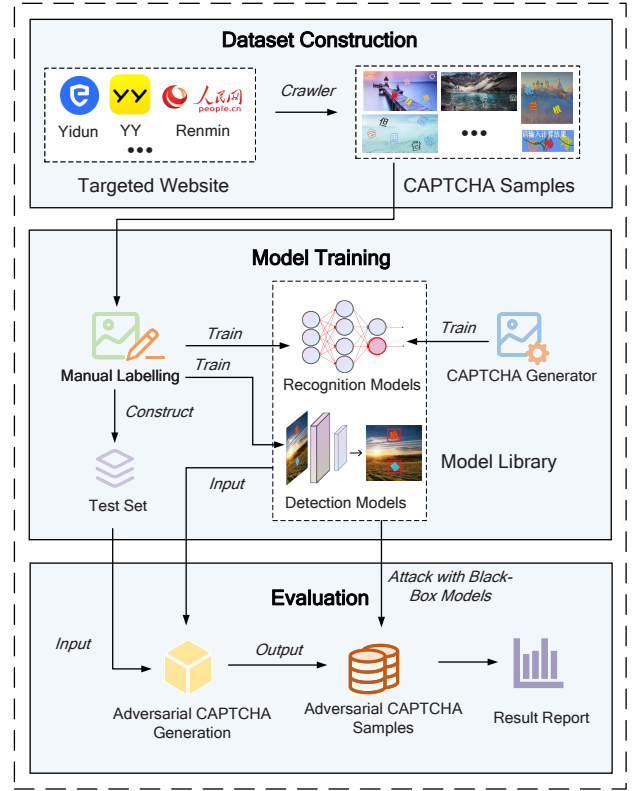


Fig. 5: Workflow of Adv-Eval Toolkit

TABLE IV: Overview of Dataset

Scheme	Total Size of Character Set	Dataset for Recognition Model		Dataset for Detection Model	
		Training	Testing	Training	Testing
Yidun	1,954	786,089	4,474	1,000	1,000
Dajie	3,651	736,200	6,000	1,000	1,000
YY	1,393	284,600	6,000	1,000	1,000
Baidu	1,234	251,800	5,000	1,000	1,000
Dingxiang	1,339	272,331	4,726	1,000	1,000
Shumei	519	105,800	2,000	1,000	500
58	1,588	322,600	5,000	1,000	1,000
Renmin	491	100,200	2,000	1,000	1,000
Sougou	20	3,280	3,281	1,000	1,000
Geetest	1,090	220,990	2,954	1,000	1,000

CAPTCHA generation. Characters in these CAPTCHAs are extracted to build training and test sets when training character recognition models. We also implemented the generator in [12] to synthesize CAPTCHA images, which expanded our training set. To form the training set, we ensure the generation of 400 characters for each character class, mixed with characters from labeled CAPTCHAs in the real world. During the construction of the character detection model, we focus solely on the locations of the characters rather than their classes. Given the relatively straightforward nature of the character detection attacks, we opt not to augment the dataset. Instead, we combine all labeled CAPTCHAs in the training set to assemble a large training set with 10,000 CAPTCHA samples. Note that for Shumei, we only collected 1,000 CAPTCHAs, so we use 500 samples as the test set and the other half as the training set. In addition, we add 500 synthesized CAPTCHAs to the training set of dataset for detection model for balance. Table IV provides detailed information about our datasets.

Model Library. We build and train four models dedicated to character recognition for each CAPTCHA scheme. They are Inception-ResNet-v2 (IncRes-v2) [59], Inception-v3 (Inc-v3) [60], ResNet-50 (Res-50) [61] and VGG-16 [62]. Regarding the detection model, we selected three widely used architectures and combined them with various feature extractors to obtain six models. They are Faster-RCNN [63] with ResNet-101, ResNet-50, and VGG-16 backbones, i.e. FRCNN (Res-101), FRCNN (Res-50), and FRCNN (VGG-16); SSD [64] with MobileNet-v2 and VGG-16 backbones, i.e. SSD (Mb-v2), SSD (VGG-16), and YOLO-v5 [65]. In addition, two SOTA attack methods designed for large character set CAPTCHAs were replicated to test the transferability of adversarial CAPTCHAs. They are the Attention-based Model [12] and the End-to-end Model utilizing Mask R-CNN [10].

V. EXPERIMENTS

A. Experimental Setup

In order to evaluate the defensive capabilities of CAPTCHAs at a finer granularity, three metrics have been defined. The lower these three metrics, the better the defensive capability: the Detection Success Rate: $DSR = D/N$, the Recognition Success Rate: $RSR = R/N$, as well as the Attack Success Rate: $ASR = A/N$, where D denotes the count of CAPTCHAs where all characters within the label are located; R denotes the count of CAPTCHAs where all characters are precisely recognized, assuming the locations are previously known; A denotes the count of CAPTCHAs that are successfully attacked, which will be defined in detail below; N denotes the total number of CAPTCHAs. Considering the different requirements for click-based and input-based CAPTCHAs, we have distinct definitions for their successful attacks. For click-based CAPTCHAs, a successful attack is defined as follows: attacking models can accurately determine the locations and classes of all characters contained in the label. For input-based CAPTCHAs, as users must accurately input all characters in the CAPTCHA, successful attacks require the model to strictly provide the characters in the label without any additional characters. In addition, for each box b within the CAPTCHA label, which corresponds to a rectangle containing characters, if there exists a box b' in the predicted results such that their Intersection over Union $IOU_{b,b'} \geq 0.5$, we consider this CAPTCHA successfully detected.

Our algorithms constrain the maximum perturbation, denoted as ϵ , to 0.1. This implies that the alteration of each pixel in CAPTCHAs is restricted to at most 10%. In ATT-VNI-CT-FGSM of the *Fine-grained Generation Module*, we configured the maximum perturbation γ induced by the attention *mask* to be 0.03. For the loss function in equation (6), we set $w_1 = w_2 = w_3 = 1, w' = 0.1$. We will empirically study the choice of w' in Section VI-B. Other parameters are the same as their original papers [26], [28]–[30], [58].

B. Security of Adversarial CAPTCHAs

We first generate adversarial CAPTCHAs with all one thousand samples in the test set of **Adv-Eval** for each of the 10 CAPTCHA schemes. We then attack both clean and

adversarial CAPTCHAs using methods under the black-box setting: (1) **SR**: applying SSD (VGG-16) [64] for character detection attack and applying Res-50 [61] for character recognition attack; (2) **YV**: applying YOLO-v5 [65] for character detection attack and applying VGG-16 [62] for character recognition attack; (3) **FA**: applying FRCNN (Res-50) [63] for character detection attack and Attention-based Model in [12] for character recognition attack. In particular, Attention-based Model [12] directly recognizes characters in the input-based schemes; (4) **EM**: applying the Mask-RCNN-based Model in [10]. For end-to-end scenarios in **FA** and **EM**, only ASR is measured. We also perform identical evaluations with the method in our conference paper [27] to show our improvement.

We make the following observations from Table V. (1) The popular attack methods can achieve high DSR, RSR and ASR when attacking clean CAPTCHAs. The most intuitive metric, ASR, lies between 6.5% and 99.0%, demonstrating the efficacy of commonly utilized attack models in successfully breaking large character set CAPTCHAs. (2) The utilization of adversarial examples leads to a substantial reduction in the three metrics. Our findings are that the existence of adversarial perturbations makes the ASR drop from 51.52% to 2.56% on average. The two SOTA attack methods [10], [12] show notable deviations in structure and parameters from white-box models; however, they are also vulnerable to the adversarial CAPTCHAs. The DSR for the majority of schemes shows a noticeable reduction. Nevertheless, the Sougou scheme exhibits a less significant decline. One possible reason is the fairly fixed character locations of this input-based CAPTCHA. Models can easily grasp the patterns of characters. Thus, misleading the attack models becomes more challenging. (3) In Table V, from the human visual perspective, the usability of these CAPTCHAs is not significantly affected since the noise in adversarial CAPTCHAs is not significant. (4) All the metrics of the attacking method obtained in our adversarial CAPTCHAs are lower than those reported in [27], which indicates our method achieves better defense. Especially, the DSR metric is significantly lower because the *Ensemble Generation Module* in this paper considers the perturbation update on characters against detection attacks. There are also improvements against recognition attacks since we further optimize characters in *Ensemble Generation Module*. To summarize, the above results show our adversarial CAPTCHAs generation method effectively reduces the black-box attack success and ensures their usability simultaneously. Meanwhile, with the improvement in *Ensemble Generation Module*, our method outperforms [27] in defending against black-box attacks.

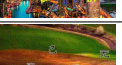
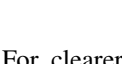
VI. DISCUSSION

A. Evaluations of Transferability

This section uses the following distinctly different schemes: Yidun (character distortion), 58 (character overlapping), Sougou (input-based scheme) and Geetest (multiple fonts).

1) *Transferability of ATT-VNI-CT-FGSM*: We first validate that our proposed ATT-VNI-CT-FGSM in *Fine-grained Generation Module* is transferable when facing the character

TABLE V: Defensive Effectiveness of Adversarial CAPTCHA under Four Black-box Attacks SR, YV, FA, EM. We compare Adversarial CAPTCHAs generated by ACG with those generated by the method in our conference paper [27].

Scheme	Clean Sample	Adversarial Sample	Metric	Clean CAPTCHA				Adversarial CAPTCHA ([27])				Adversarial CAPTCHA (Ours)			
				SR [61], [64]	YV [62], [65]	FA [12]	EM [10]	SR [61], [64]	YV [62], [65]	FA [12]	EM [10]	SR [61], [64]	YV [62], [65]	FA [12]	EM [10]
Yidun			DSR	95.5%	86.4%	91.3%	—	39.5%	31.7%	28.3%	—	10.6%	6.6%	6.5%	—
			RSR	50.2%	22.6%	33.7%	—	0.0%	0.0%	0.1%	—	0.0%	0.0%	0.0%	—
			ASR	43.8%	20.1%	24.0%	6.5%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%
Dajie			DSR	99.6%	99.8%	99.4%	—	50.4%	33.7%	50.8%	—	17.2%	11.4%	34.7%	—
			RSR	57.1%	19.0%	38.1%	—	0.0%	0.0%	0.1%	—	0.0%	0.0%	0.0%	—
			ASR	42.5%	10.4%	21.3%	26.8%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%
YY			DSR	99.2%	97.7%	99.2%	—	27.1%	33.0%	65.8%	—	0.9%	0.5%	8.8%	—
			RSR	70.7%	5.5%	57.6%	—	0.0%	0.0%	0.0%	—	0.0%	0.0%	0.0%	—
			ASR	77.5%	8.2%	58.9%	21.2%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%
Baidu			DSR	99.8%	99.2%	99.4%	—	35.4%	34.1%	26.4%	—	5.1%	2.6%	8.4%	—
			RSR	87.8%	88.4%	65.9%	—	0.0%	0.0%	0.0%	—	0.0%	0.0%	0.0%	—
			ASR	83.0%	78.3%	57.0%	60.1%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%
Dingxiang			DSR	98.8%	98.2%	97.7%	—	26.9%	21.7%	5.2%	—	0.2%	0.0%	0.0%	—
			RSR	53.5%	25.3%	40.3%	—	0.0%	0.0%	0.0%	—	0.0%	0.0%	0.0%	—
			ASR	57.0%	30.5%	40.1%	21.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%
Shumei			DSR	94.2%	79.8%	91.4%	—	72.2%	62.8%	66.0%	—	48.4%	45.0%	51.0%	—
			RSR	71.0%	34.0%	65.8%	—	0.4%	0.0%	7.0%	—	0.2%	0.0%	0.8%	—
			ASR	75.2%	49.0%	68.4%	88.2%	1.8%	1.2%	9.0%	5.4%	0.2%	0.0%	0.8%	0.2%
58			DSR	98.7%	95.8%	96.7%	—	19.9%	19.2%	2.6%	—	3.5%	2.2%	1.2%	—
			RSR	66.3%	32.1%	40.3%	—	0.1%	0.0%	0.0%	—	0.0%	0.0%	0.0%	—
			ASR	66.6%	27.5%	38.3%	49.5%	0.0%	0.0%	0.0%	0.9%	0.0%	0.0%	0.0%	0.0%
Renmin			DSR	100.0%	99.6%	—	—	47.2%	84.4%	—	—	19.3%	45.7%	—	—
			RSR	73.6%	27.5%	—	—	3.5%	1.3%	—	—	4.6%	1.4%	—	—
			ASR	70.7%	21.2%	66.2%	38.3%	1.6%	0.1%	7.5%	1.9%	0.5%	0.2%	6.3%	1.4%
Sougou			DSR	98.5%	98.5%	—	—	95.8%	92.1%	—	—	92.2%	82.5%	—	—
			RSR	99.3%	89.2%	—	—	6.2%	32.0%	—	—	4.5%	30.6%	—	—
			ASR	99.0%	91.2%	87.2%	78.2%	9.5%	36.9%	39.3%	21.1%	6.4%	29.1%	37.7%	18.9%
Geetest			DSR	99.9%	99.9%	100.0%	—	85.8%	95.4%	97.2%	—	42.9%	44.1%	74.0%	—
			RSR	94.2%	28.7%	64.0%	—	1.5%	2.6%	1.6%	—	0.3%	1.4%	0.3%	—
			ASR	93.9%	27.2%	59.4%	77.3%	1.2%	0.7%	0.9%	0.8%	0.2%	0.2%	0.0%	0.1%

recognition attacks. For clearer comparison, the recognition metric here is the top-1 accuracy. For each scheme from the test set of **Adv-Eval**, 1,000 characters were randomly selected for adversarial character generation. Next, we attack these characters with IncRes-v2 (white-box model), Inc-v3, Res-50, VGG-16, ensemble models averaging the prediction probabilities of Inc-v3, Res-50, VGG-16, and the Attention-based Model [12]. We denote these models as **R1**, **R2**, **R3**, **R4**, **R5** and **R6** respectively. Four methods are leveraged as baselines to compare the transferability: VNI-CT-FGSM [29] with *Attention Mechanism* in our method removed, VNI-FGSM [29] with Input Transformations removed, the basic FGSM [25] and another classical adversarial example generation method DeepFool [66] respectively. We also compare clean characters and characters with random noise from the normal distribution clipped by ϵ . Table VII shows examples of how adversarial CAPTCHAs generated by ACG mislead multiple black-box recognition models. As indicated by the underlined characters, many of them are misrecognized as entirely different characters, while the few normal characters suggest that the attacked models still have some chance of correctly recognizing them. Furthermore, we make the following observations from Table VI. (1) High accuracy can be achieved by recognition models when attacking CAPTCHAs without adversarial noise. (2) Our approach significantly reduces the accuracy of multiple recognition models by generating adversarial characters through the white-box model. (3) The transferability of adversarial characters can be enhanced by the strategies in *Fine-grained Generation Module*. Therefore, our method outperforms other methods when defending against all

the black-box models after integrating these strategies.

2) *Transferability against Character Detection Attacks*: We generate global perturbations on the whole CAPTCHA image with characters already perturbed in this section. As for the detection model, we then use three white-box models, i.e., FRCNN (Res-101), SSD (Mb-v2), and YOLO-v5, which are referred to as **D1**, **D2**, and **D3** and three black-box models, i.e., FRCNN(Res-50), FRCNN(VGG-16), and SSD(VGG-16), which are referred to as **D4**, **D5**, and **D6** respectively. Our experiment is conducted on CAPTCHAs and three different types of noise, i.e., (1) random noise from the normal distribution, (2) perturbations generated only on the background with the method in [27], (3) our method in Section III-C adding global perturbation. Both [27] and our method use SVRE-MI-FGSM [58] for optimization. Next, the perturbed test set is fed into detection models, and the DSR is measured.

Table VIII shows some examples of defending against character detection attacks. We observe that only a small number of predicted bounding boxes (green boxes) correctly localize the characters, while a large number of boxes are deviated due to adversarial perturbations (red boxes), ultimately causing parts of characters as well as background regions to be incorrectly predicted as ‘character’. In addition, although some boxes correctly enclose the characters, their boundaries may be expanded due to noise, resulting in the characters occupying only a small portion of the bounding boxes, which also reduces the effectiveness of recognition models. From Table IX, our observations are as follows. (1) The DSR of character detection models is high when attacking both clean and randomly perturbed CAPTCHAs. It demonstrates

TABLE VI: Transferability Evaluation of Adversarial Characters against Recognition Models. R1, R2, R3, R4, R5, R6 denote IncRes-v2 [59], Inc-v3 [60], Res-50 [61], VGG-16 [62], Ensemble model [59]–[62], Attention-based Model in [12]. A $\rightarrow$ B denotes applying adversarial CAPTCHA generated by model A to defend attacks from model B.

Scheme	Generation Method	Character Recognition Attack					
		White-box Model	Black-box Model				
		R1 $\rightarrow$ R1	R1 $\rightarrow$ R2	R1 $\rightarrow$ R3	R1 $\rightarrow$ R4	R1 $\rightarrow$ R5	R1 $\rightarrow$ R6
Yidun	Clean Character	90.4%	82.3%	81.7%	64.0%	93.1%	76.3%
	Random Noise	85.9%	73.5%	72.6%	54.3%	87.2%	69.4%
	DeepFool [66]	35.0%	78.9%	78.9%	63.0%	89.1%	69.3%
	FGSM [25]	12.1%	21.8%	22.3%	24.7%	27.4%	40.6%
	VNI-FGSM [29]	0.4%	14.1%	17.9%	21.9%	11.7%	39.8%
	VNI-CT-FGSM [29]	0.1%	0.8%	1.6%	1.1%	0.1%	17.6%
	Our Method	0.0%	0.0%	0.3%	0.5%	0.0%	6.0%
58	Clean Character	93.6%	93.1%	91.5%	77.0%	96.5%	81.8%
	Random Noise	91.1%	84.7%	90.5%	75.0%	95.6%	80.9%
	DeepFool [66]	11.7%	92.2%	91.3%	76.9%	95.9%	81.4%
	FGSM [25]	1.3%	15.0%	37.1%	41.3%	46.0%	51.6%
	VNI-FGSM [29]	0.0%	6.0%	25.1%	30.6%	25.5%	37.6%
	VNI-CT-FGSM [29]	0.0%	2.2%	4.2%	4.8%	5.4%	8.3%
	Our Method	0.0%	0.8%	2.3%	3.0%	3.0%	4.6%
Sougou	Clean Character	99.9%	99.5%	100.0%	97.1%	99.9%	82.5%
	Random Noise	96.1%	97.4%	75.8%	96.2%	99.0%	73.2%
	DeepFool [66]	49.9%	99.5%	100.0%	97.0%	99.9%	82.5%
	FGSM [25]	16.5%	79.9%	39.5%	91.5%	92.0%	47.6%
	VNI-FGSM [29]	0.0%	84.1%	56.8%	91.9%	93.0%	55.2%
	VNI-CT-FGSM [29]	22.9%	49.7%	40.4%	71.2%	72.4%	40.7%
	Our Method	17.1%	43.8%	34.5%	67.6%	69.2%	34.9%
Geetest	Clean Character	98.3%	94.5%	97.9%	61.6%	98.2%	85.0%
	Random Noise	97.7%	93.1%	97.6%	60.3%	98.2%	82.2%
	DeepFool [66]	47.8%	93.3%	98.0%	61.3%	98.0%	83.7%
	FGSM [25]	16.6%	54.0%	76.8%	48.4%	62.2%	46.9%
	VNI-FGSM [29]	1.3%	47.4%	78.0%	46.5%	47.1%	51.0%
	VNI-CT-FGSM [29]	2.3%	9.6%	22.8%	17.8%	14.5%	11.3%
	Our Method	1.2%	7.5%	10.7%	17.9%	7.2%	9.0%

TABLE VII: Examples of Defending Multiple Recognition Attacks. Underlined text denotes errors, and normal text denotes correctness.

Scheme	Sample	Correct Label	Incorrect Results of Recognition					
			R1	R2	R3	R4	R5	R6
Yidun		‘贴’	‘震’	‘慰’	‘遭’	‘掌’	‘震’	‘忆’
58		‘色’	‘龟’	‘龟’	‘色’	‘创’	‘色’	‘色’
Sougou		‘八’	‘1’	‘1’	‘1’	‘六’	‘六’	‘八’
Geetest		‘鸭’	‘襄’	‘奥’	‘宿’	‘鸭’	‘鸭’	‘鸭’

TABLE VIII: Examples of Defending Multiple Detection Attacks. **Red** denotes errors, and **green** denotes correctness.

Scheme	Incorrect Results of Detection					
	D1	D2	D3	D4	D5	D6
Yidun						
58						
Sougou						
Geetest						

that character detection models rarely make mistakes when attacking CAPTCHAs without adversarial perturbations. (2) Adversarial CAPTCHAs, when subjected to character detection attacks, result in lower DSR. This implies that our defense reduces the effectiveness of both black-box and white-box character detection models. Therefore, the models cannot

fully locate all the characters in the adversarial CAPTCHAs. The results indicate that by combining various loss functions from different architectures, the ensemble method can produce adversarial CAPTCHAs that are transferable across detection attack models. (3) Utilizing ensemble generation for global loss, which is different from the approach described in [27], can further enhance defense capabilities against character detection models since it also considers the perturbation update on the characters. We also observe that the detection model still has a relatively high DSR in the adversarial version of Sougou CAPTCHAs. We analyze that since Sougou is an input-based CAPTCHA scheme and the locations of characters are very fixed. It is natural for the detection model to learn the characteristic that the characters are arranged in a line.

B. Choice of Recognition Loss Weight W'

Since our key insight in Section III-C is the ensemble optimization of recognition and detection, we now study the influence of W' in Equation (6). Specifically, we generate adversarial CAPTCHAs with W' chosen from $\{0, 0.001, 0.01, 0.1, 1\}$. We also use four representative schemes, Yidun, 58, Sougou, and Geetest, to generate 1,000 adversarial CAPTCHAs from their test sets, respectively. To show the general trend, we use all the detection and recognition models in **Adv-Eval** to attack the generated adversarial CAPTCHAs.

From the experimental results, we have the following observations. (1) Fig. 6 shows the accuracy trends of recognition models. The accuracy declines with increasing recognition loss weight W' , which is natural because when the recognition loss has a larger proportion, the optimization magnitude is

TABLE IX: Transferability Evaluation of Adversarial CAPTCHA against Detection Models. D1, D2, D3, D4, D5, D6 denote FRCNN (Res-101) [63], SSD (Mb-v2) [64], YOLO-v5 [65], FRCNN(Res-50), FRCNN(VGG-16), SD(VGG-16) respectively. $A \rightarrow B$ denotes applying adversarial CAPTCHAs generated by model A to defend attacks from model B.

Scheme	Attack	Character Detection Attack					
		White-box Model			Black-box Model		
		D1+D2+D3 $\rightarrow$ D1	D1+D2+D3 $\rightarrow$ D2	D1+D2+D3 $\rightarrow$ D3	D1+D2+D3 $\rightarrow$ D4	D1+D2+D3 $\rightarrow$ D5	D1+D2+D3 $\rightarrow$ D6
Yidun	Clean CAPTCHA	94.1%	76.4%	86.4%	91.3%	94.8%	95.5%
	Random Noise	91.0%	74.2%	87.4%	88.3%	89.6%	91.8%
	SVRE-MI-FGSM [27]	7.7%	1.3%	31.7%	28.3%	30.7%	39.5%
	Our Method	2.8%	0.0%	6.6%	6.5%	10.0%	10.6%
58	Clean CAPTCHA	97.3%	79.2%	95.8%	96.7%	99.0%	98.7%
	Random Noise	84.9%	75.7%	84.4%	82.3%	84.1%	91.5%
	SVRE-MI-FGSM [27]	1.3%	5.6%	19.2%	2.6%	14.8%	19.9%
	Our Method	0.5%	0.0%	2.2%	1.2%	2.9%	3.5%
Sougou	Clean CAPTCHA	97.5%	73.4%	98.5%	97.9%	98.3%	98.5%
	Random Noise	97.4%	74.5%	98.3%	98.2%	98.1%	98.5%
	SVRE-MI-FGSM [27]	93.0%	70.2%	92.1%	93.5%	94.0%	95.8%
	Our Method	83.9%	54.0%	82.5%	87.2%	89.1%	92.2%
Geetest	Clean CAPTCHA	100.0%	99.9%	99.9%	100.0%	100.0%	99.9%
	Random Noise	100.0%	99.9%	99.8%	100.0%	100.0%	99.9%
	SVRE-MI-FGSM [27]	86.6%	0.1%	95.4%	97.2%	83.6%	85.8%
	Our Method	37.6%	0.0%	44.1%	74.0%	39.9%	42.9%

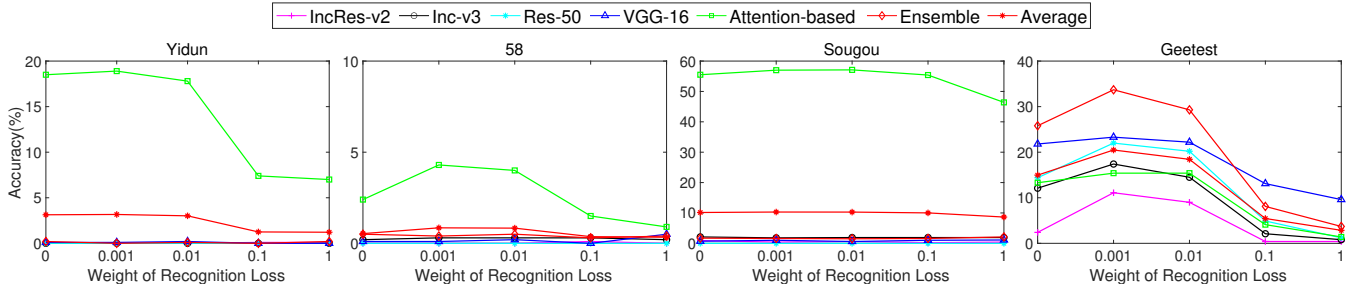


Fig. 6: The Accuracy of Attack Models in Adversarial CAPTCHAs Generated by Different Recognition Loss Weight W' .

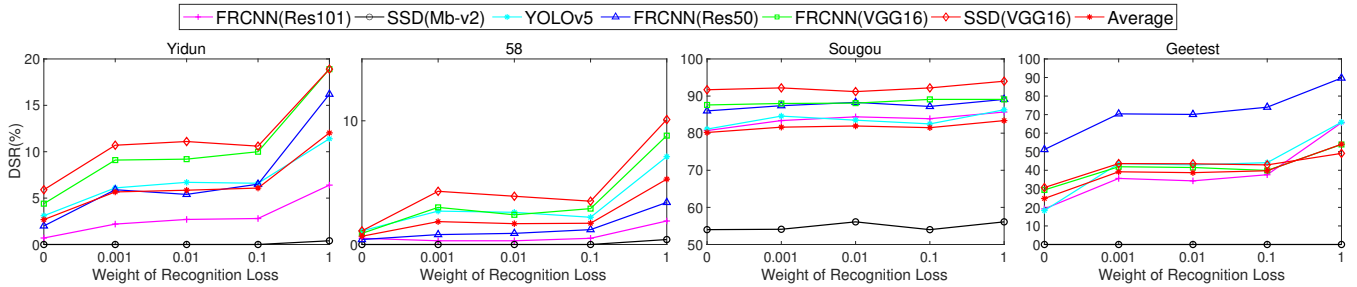


Fig. 7: The DSR of Attack Models in Adversarial CAPTCHAs Generated by Different Recognition Loss Weight W'

larger for the recognition part, which is similar for all the attack models and CAPTCHA schemes. (2) The DSR trends of detection models are shown in Fig. 7. In contrast to accuracy, the DSR increases overall with increasing W' since the detection loss has little proportion when W' is large. Interestingly, after a small peak at 0.001, the DSRs decrease or remain stable until 0.1. We infer that adding optimization to recognition loss can increase the generalizability of adversarial CAPTCHAs. Thus, the appropriate selection of the recognition loss weight W' is also helpful for the defense of detection attacks. (3) We can also see that after W' is reduced to 0.1, recognition accuracy can maintain a stable and low level. Since the low point of DSR is also 0.1, we thus conclude that $W' = 0.1$ is a suitable parameter balancing the defense against recognition and detection.

C. Evaluation of Robustness

In this section, we evaluate the robustness of our adversarial CAPTCHAs considering two countermeasures that attackers can leverage: *Adversarial Training* and *Filters*. For each CAPTCHA scheme, we use adversarial CAPTCHAs generated on 1,000 samples in the test set of **Adv-Eval** for evaluation.

1) *Robustness under Adversarial Training*: Once attackers discover the adversarial perturbations on CAPTCHAs, they can use more advanced methods against adversarial examples to crack adversarial CAPTCHAs. Thus, we consider adversarial training [67], which is a powerful countermeasure utilizing adversarial examples for model training. We assume the same scenarios as in [6] that attackers only access a restricted number of adversarial CAPTCHAs and fine-tune the attack models, which have been trained on normal CAPTCHA datasets. Next,

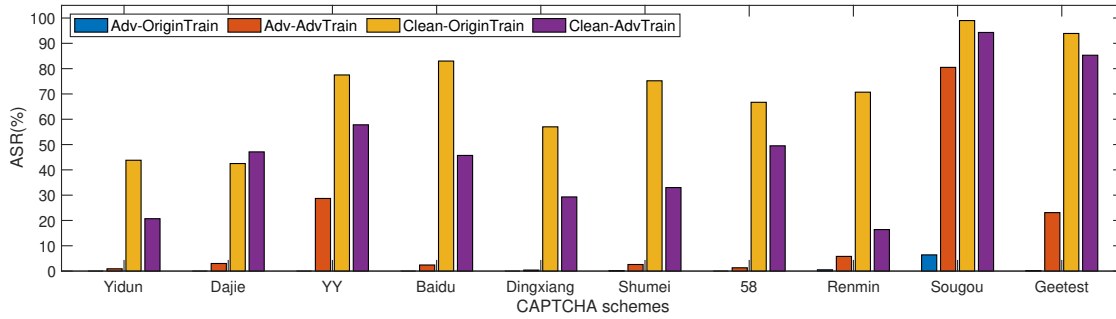


Fig. 8: ASR in attacking 10 CAPTCHA schemes with SSD(VGG-16)+ResNet-50 in four settings: (1) Adv-OriginTrain: attacking adversarial CAPTCHA with original models without adversarial training; (2) Adv-AdvTrain: attacking adversarial CAPTCHAs with adversarially trained models; (3) Clean-OriginTrain: attacking clean CAPTCHAs with original models without adversarial training; (4) Clean-AdvTrain: attacking clean CAPTCHAs with adversarially trained models.

ResNet-50 in the Model Library is fine-tuned with characters in 1,000 adversarial CAPTCHAs generated from the training set. We also fine-tuned the SSD (VGG-16) for character detection, leveraging 1,000 adversarial CAPTCHAs generated from the training set for fine-tuning.

As shown in Fig. 8, adversarial training can improve the ASR of attack models by comparing the results in Adv-OriginTrain (nearly 0%) and Adv-AdvTrain. However, the ASR of Adv-AdvTrain is still significantly lower than Clean-OriginTrain's, i.e., the effect of undertaking adversarial training is limited. The ASR is still much lower than that of Clean-OriginTrain, i.e. using original models without adversarial training to attack clean CAPTCHAs. Also, the ASR of some complex schemes is still at a significantly low level (nearly 0%). Therefore, adversarial training is not a general countermeasure against the ACG framework. From the comparison between Clean-AdvTrain and Clean-OriginTrain, we observe that adversarial training decreases the ASR of models against clean CAPTCHAs.

2) *Robustness under Various Filters*: Some research indicates that image preprocessing techniques can reduce the defense effectiveness of adversarial CAPTCHAs [35], [36]. The basic assumption behind these approaches is that adversarial perturbations consist mainly of high-frequency noise, which is hardly perceptible to humans. Therefore, a natural defense strategy is to apply spatial filtering as a preprocessing step before feeding images into the classifier, in order to “denoise” the input and weaken adversarial perturbations. This strategy is model-agnostic since it can be applied as an external wrapper without changing the target model itself. Motivated by this, we investigate whether common image filters can also reduce the effectiveness of our adversarial CAPTCHA defense.

Filtering Pipeline. We design the following pipeline: given an adversarial CAPTCHA, we first apply a filter from the Pillow library², then feed the filtered image into the SR attack pipeline, which includes SSD (VGG-16) for character detection and ResNet-50 for character recognition. In this way, we can evaluate whether preprocessing weakens the

robustness of adversarial CAPTCHAs. We select five widely used filters following [36]: (1) BLUR performs simple mean smoothing by averaging pixel intensities in local neighborhoods; (2) GaussianBlur applies a Gaussian-weighted kernel that reduces high-frequency noise with smoother transitions; (3) MedianFilter replaces each pixel with the neighborhood median, effectively removing salt-and-pepper noise while preserving edges; (4) MaxFilter takes the maximum value within a neighborhood, enhancing bright regions and it resembles morphological dilation; and (5) SMOOTH applies a mild smoothing kernel that reduces fine details but with a weaker effect than GaussianBlur.

Impact on Robustness. Despite mitigating some perturbations, these filters inevitably distort the image content. On the one hand, they may remove high-frequency adversarial perturbations and thereby improve the attack success rate against adversarial CAPTCHAs (e.g., MedianFilter for YY and Renmin CAPTCHAs, GaussianBlur for Baidu). On the other hand, they also destroy useful structural information in the CAPTCHA itself, which can harm the recognition accuracy (e.g., SMOOTH sometimes decreases detection success). Our experimental results (Table X) show that while certain filters lead to marginal improvements in the DSR or RSR of attack models, no single filter consistently increases robustness across all schemes. As a result, the overall increase in ASR remains very limited (the maximum improvement being from 1.6% to 3.0%). We therefore conclude that our adversarial CAPTCHAs are robust against most widely used image filters.

D. Comparison with Traditional Defense Methods

To evaluate the defensive effectiveness of adversarial CAPTCHAs relative to traditional defenses, we selected three classical defense techniques in Gao et al. [68]: (1) Distortion: applying random geometric transformations to characters so that their strokes deviate from regular straight or curved shapes. (2) Lines & Spots: overlaying randomly generated interfering lines and noise points on the image, thereby blurring the boundary between foreground characters and the background. (3) Overlap: partially overlapping adjacent

²<https://pillow.readthedocs.io/en/stable/>

TABLE X: The ASR of SR (SSD(VGG-16)+ResNet-50) Pipeline when Attacking Adversarial CAPTCHA Processed by Filters.

Filter	Metric	Scheme									
		Yidun	Dajie	YY	Baidu	Dingxiang	Shumei	58	Renmin	Sougou	Geetest
None	DSR	39.5%	50.4%	27.1%	35.4%	26.9%	72.2%	19.9%	47.2%	95.8%	85.8%
	RSR	0.0%	0.0%	0.0%	0.0%	0.0%	0.4%	0.1%	3.5%	6.2%	1.5%
	ASR	0.0%	0.0%	0.0%	0.0%	0.0%	1.8%	0.0%	1.6%	9.5%	1.2%
BLUR	DSR	2.5%	2.1%	9.5%	42.8%	2.8%	3.6%	0.0%	1.8%	39.8%	56.4%
	RSR	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.6%
	ASR	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.9%
GaussianBlur	DSR	0.1%	0.0%	12.9%	47.1%	2.8%	0.0%	0.0%	0.2%	19.0%	53.3%
	RSR	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.2%
	ASR	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.6%
MedianFilter	DSR	33.5%	16.1%	31.4%	6.4%	2.1%	51.0%	2.8%	56.1%	93.7%	90.0%
	RSR	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	5.6%	0.0%	0.6%
	ASR	0.0%	0.0%	0.0%	0.0%	0.0%	0.2%	0.0%	2.0%	0.3%	0.6%
MaxFilter	DSR	24.1%	11.9%	38.2%	23.9%	6.7%	58.0%	5.4%	26.7%	85.5%	95.0%
	RSR	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.1%	0.2%	2.1%
	ASR	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	1.4%
SMOOTH	DSR	23.1%	16.7%	20.7%	11.4%	1.5%	49.0%	1.7%	65.5%	91.8%	87.0%
	RSR	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	6.8%	0.0%	0.5%
	ASR	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	3.0%	0.0%	0.5%

TABLE XI: Performance Comparison between Adversarial CAPTCHAs and Traditional Defense Methods. The underlined results indicate the performance of recognition models fine-tuned on CAPTCHAs with traditional defense techniques.

Method	Sample	Label	SR [61], [64]	YV [62], [65]
Clean		‘沈’	91.7% / –	93.2% / –
Distortion		‘共’	71.2% / <u>83.1%</u>	66.6% / <u>86.4%</u>
Lines & Spots		‘阱’	49.7% / <u>77.1%</u>	44.5% / <u>83.7%</u>
Overlap		‘空’	40.9% / <u>70.2%</u>	41.4% / <u>76.9%</u>
Ours		‘巧’	0.0% / <u>3.5%</u>	0.0% / <u>5.7%</u>

characters such that the outer contours of individual characters become incomplete.

Since most CAPTCHAs used in prior experiments already incorporate some form of traditional defense, we select Baidu CAPTCHAs without such processing for our evaluation. As traditional defenses primarily affect recognition models, we evaluate and report the RSR under SR and YV attack pipelines. To control variables, all experiments are conducted on synthetic test sets with 1,000 samples. Results in Table XI show that applying these three defense methods reduces the RSR of recognition models. However, after fine-tuning with the adversarial training setting described in Section VI-C1, performance is almost fully recovered. In contrast, the RSR against adversarial CAPTCHAs remains consistently low, indicating that our method provides stronger defensive robustness.

E. Evaluation of Usability

This section evaluates the usability of adversarial CAPTCHAs with samples generated by the ACG framework. For each of the 10 CAPTCHA schemes in the Adv-Eval toolkit, we randomly selected 10 images from the test set and applied perturbation magnitudes $\epsilon \in \{0\%, 1\%, 5\%, 10\%, 20\%\}$, resulting in 500 adversarial CAPTCHAs in total. We recruited 25 volunteers (university students and staff, aged 17–30, with normal vision and no

color blindness) to participate in an anonymous online study. Each participant was asked to solve 20 CAPTCHA images, with four samples drawn from each perturbation level. To avoid bias from repeated exposure, different perturbation versions of the same base CAPTCHA image were never assigned to the same participant. For each image, participants were asked to transcribe the characters they recognized. All participants joined the study voluntarily and provided informed consent. The study was conducted anonymously to protect participants' privacy. We then compared their responses against ground-truth labels and calculated the ASR. In parallel, we applied the **SR** attacking pipeline to the same set of images for comparison. As shown in Fig. 9, both users and the attack model exhibit a decreasing ASR as ϵ increases. However, for $\epsilon \geq 5\%$, human ASRs remain consistently higher than those of the **SR** attack, with human ASRs above 60% at $\epsilon = 5\%$ and above 30% at $\epsilon = 10\%$, while **SR**'s ASRs approach 0%. These results suggest that a perturbation level of around $\epsilon = 10\%$ achieves a good balance between defensive effectiveness and usability in practice.

F. Future Work

Although ACG can effectively defend against attacks on large character set CAPTCHAs, limitations still exist. Next, we analyze the limitations and discuss future work.

Combination with Behavior Verification. Some CAPTCHA schemes such as Google's reCAPTCHA-v3³ have been designed to analyze users' behaviors, including the mouse trajectory and click frequency. Although the ACG framework focuses on the CAPTCHA generation algorithm, it can potentially integrate with behavior verification systems. For example, we can extend the ACG framework to generate targeted adversarial examples that mislead models to give our expected recognition result. When users frequently give the wrong but expected answer, we can identify the user as an automated attack program. Moreover, the detection attacks can also be

³<https://developers.google.com/recaptcha/docs/v3>

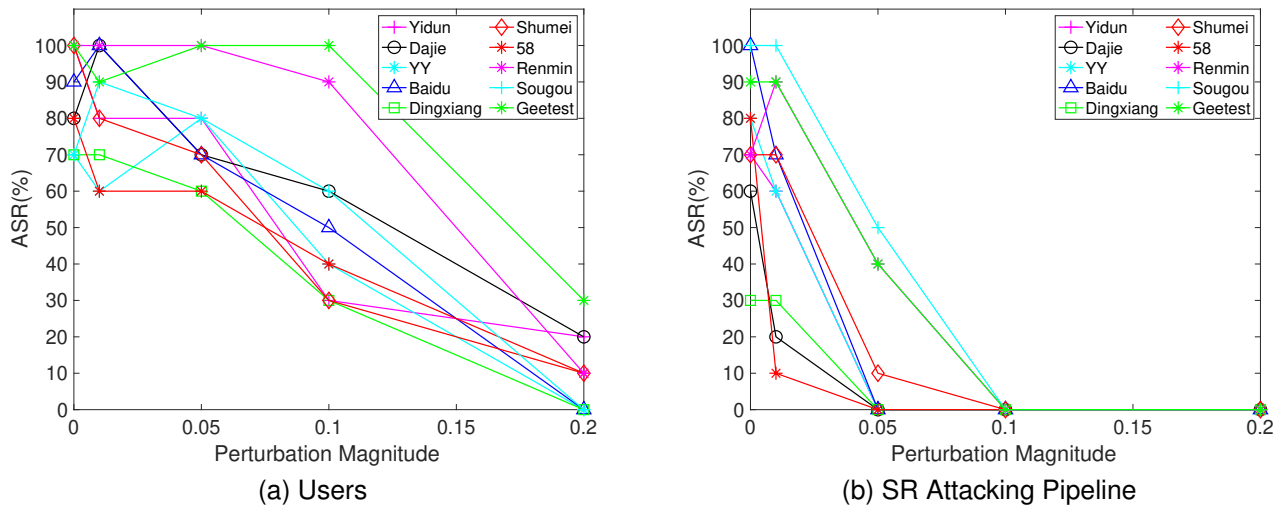


Fig. 9: ASR Metric in Solving 10 Adversarial CAPTCHA Schemes, under the Setting of $\epsilon = 0\%, 1\%, 5\%, 10\%, 20\%$

affected by the targeted adversarial examples. Furthermore, the ACG framework can be fine-tuned using large amounts of users' answers. Shi et al. [18] have explored such fine-tuning schemes in text CAPTCHAs. Therefore, it is interesting to combine ACG with behavior verification.

Efficient Generation of Adversarial CAPTCHAs. Even though the ACG framework can generate large batches of adversarial CAPTCHAs on the server side, the complex generation process may hinder the customized design of large character set CAPTCHAs. Moreover, the white-box models need to be updated when generating adversarial CAPTCHA for a new scheme. Therefore, both the adversarial CAPTCHA generation and white-box fine-tuning can be accelerated while providing a similar level of security in this paper.

Reinforcement of Input-Based CAPTCHA. ACG framework performs well in most schemes; however, for input-based CAPTCHAs, its performance is suboptimal under detection attacks. We believe this is due to two types of input-based CAPTCHA schemes, namely Renmin and Sougou, where the background occupies a relatively small proportion, and the character positions are fixed. This feature is easily learned by the model, and the adversarial example generation methods and parameters that perform well on click-based schemes do not adapt to input-based schemes. Therefore, it is necessary to automatically search parameters for different schemes based on the expected performance and user experience. Additionally, if adversarial patch techniques [69], [70] specifically designed for object detectors are introduced, it may alleviate the performance degradation of ACG in this scenario.

Adversarial Defense against VLM-based Attacks. Recent studies show that both generalist solvers (e.g., Halligan [71]) and task-specific agents (e.g., MCA-Bench [72]) pose systematic threats to existing CAPTCHA schemes. Halligan demonstrates how zero-shot VLM agents can transform CAPTCHA solving into a heuristic search problem over graphical user interface action spaces, while MCA-Bench confirms that fine-

tuned models can achieve near-perfect success rates on specific CAPTCHA families. Meanwhile, IllusionCAPTCHA [73] highlights the feasibility of leveraging perceptual illusions to exploit cognitive gaps between humans and VLMs. These findings collectively inspire several future research directions closely related to our adversarial CAPTCHA generation framework. First, *adversarial noise for jailbreak* can be further explored: by embedding imperceptible perturbations resembling prompt-injection cues, we can mislead the language interface of VLMs, causing them to hallucinate or deviate from the intended task objectives. Second, targeted perturbations can be designed to specifically mislead the *vision tower* of VLMs, disrupting their localization functions by drawing attention to irrelevant background textures, while maintaining readability for humans. Finally, inspired by semantic and structural puzzles, Chinese CAPTCHAs can integrate *radical/tone-aware tasks*, homophone-based ordering, or compositional glyph reasoning, which remain simple and intuitive for literate human users but interfere with the symbolic grounding of VLMs. These strategies extend the ACG framework toward a more comprehensive adversarial defense against the next generation of multimodal attackers.

Incorporate Advanced Adversarial Example Techniques. Future research on the ACG framework should move beyond traditional accuracy-based defenses to address more covert and diverse adversarial attacks. On one hand, inspired by semantic patch attacks like AdvLogo [74], the framework could be upgraded from adding diffuse noise to generating structured, semantic perturbations that integrate seamlessly with the CAPTCHA's background. This would defend against more advanced attacks while simultaneously improving usability and protecting the user experience. The Overload [75] framework introduces a novel attack vector: latency. Its objective is not to cause model misclassification but to significantly increase the model's inference time. The loss function in ACG for generating global noise could incorporate a penalty term

for strategies that induce system latency, such as those that generate a massive number of candidate bounding boxes.

VII. CONCLUSION

To secure the widely employed CAPTCHAs with a large character set, we design a framework named **ACG**. It first generates adversarial perturbations on the individual characters with a fine-grained method. Subsequently, **ACG** generates global perturbations on the whole CAPTCHA image with a novel multi-task loss function of the recognition and detection model. Then, we collect and generate popular Chinese CAPTCHAs to construct datasets and train multiple models. These resources form our **Adv-Eval** toolkit, which generates and evaluates adversarial CAPTCHAs. We open-source our **Adv-Eval** toolkit to promote the security research on CAPTCHAs. Furthermore, we conduct experiments under **Adv-Eval**. The results demonstrate that adversarial CAPTCHAs generated by the **ACG** framework have strong effectiveness, transferability, robustness, and usability.

Moreover, the **ACG** framework is adaptable to other large character set languages such as Japanese and Korean. Since CAPTCHA recognition is a multi-label classification task, the class scale and sample complexity in these languages are comparable to those in Chinese, indicating good generalizability [76]. The main challenge lies in dataset availability: while diverse Chinese CAPTCHAs are accessible, collecting and annotating sufficient data for other languages may be more difficult. However, we can transfer the capability by fine-tuning existing models trained on Chinese CAPTCHAs.

ACKNOWLEDGMENTS

This work is partially supported by the Key Research and Development Program of Science and Technology Department of Sichuan Province under grant No. 2023YFG0145 and National Key Research and Development Program of China under grant No. 2022YFC3303101.

REFERENCES

- [1] M. Torky, A. Meligy, and H. Ibrahim, "Securing online social networks against bad bots based on a necklace captcha approach," in *Proceedings of the 12th International Computer Engineering Conference*, 2016, pp. 158–163.
- [2] B. B. Zhu, J. Yan, G. Bao, M. Yang, and N. Xu, "Captcha as graphical passwords—a new security primitive based on hard ai problems," *IEEE Transactions on Information Forensics and Security*, vol. 9, no. 6, pp. 891–904, 2014.
- [3] M. Guerar, M. Migliardi, A. Merlo, M. Benmohammed, F. Palmieri, and A. Castiglione, "Using screen brightness to improve security in mobile social network access," *IEEE Transactions on Dependable and Secure Computing*, vol. 15, no. 4, pp. 621–632, 2016.
- [4] Y. Zi, H. Gao, Z. Cheng, and Y. Liu, "An end-to-end attack on text captchas," *IEEE Transactions on Information Forensics and Security*, vol. 15, pp. 753–766, 2019.
- [5] M. Tang, H. Gao, Y. Zhang, Y. Liu, P. Zhang, and P. Wang, "Research on deep learning techniques in breaking text-based captchas and designing image-based captcha," *IEEE Transactions on Information Forensics and Security*, vol. 13, no. 10, pp. 2522–2537, 2018.
- [6] P. Wang, H. Gao, X. Guo, Z. Yuan, and J. Nian, "Improving the security of audio captchas with adversarial examples," *IEEE Transactions on Dependable and Secure Computing*, pp. 1–18, 2023.
- [7] Y. Xu, G. Reynaga, S. Chiasson, J.-M. Frahm, F. Monrose, and P. C. van Oorschot, "Security and usability challenges of moving-object captchas: decoding codewords in motion," in *Proceedings of the 21st USENIX Security Symposium*, 2012, pp. 49–64.
- [8] B. Zhao, H. Weng, S. Ji, J. Chen, T. Wang, Q. He, and R. Beyah, "Towards evaluating the security of real-world deployed image captchas," in *Proceedings of the 11th ACM Workshop on Artificial Intelligence and Security*, 2018, pp. 85–96.
- [9] C. Li, X. Chen, H. Wang, P. Wang, Y. Zhang, and W. Wang, "End-to-end attack on text-based captchas based on cycle-consistent generative adversarial network," *Neurocomputing*, vol. 433, pp. 223–236, 2021.
- [10] J. Nian, P. Wang, H. Gao, and X. Guo, "A deep learning-based attack on text captchas by using object detection techniques," *IET Information Security*, vol. 16, no. 2, pp. 97–110, 2022.
- [11] W. Aiken and H. Kim, "Poster: Deepcrack: Using deep learning to automatically crack audio captchas," in *Proceedings of the 13th Asia Conference on Computer and Communications Security*, 2018, pp. 797–799.
- [12] P. Wang, H. Gao, Q. Rao, S. Luo, Z. Yuan, and Z. Shi, "A security analysis of captchas with large character sets," *IEEE Transactions on Dependable and Secure Computing*, vol. 18, no. 6, pp. 2953–2968, 2020.
- [13] J. Wang, Y. Yang, J. Mao, Z. Huang, C. Huang, and W. Xu, "Cnn-rnn: A unified framework for multi-label image classification," in *Proceedings of the 25th IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2016, pp. 2285–2294.
- [14] A. Algwil, D. Ciresan, B. Liu, and J. Yan, "A security analysis of automated chinese turing tests," in *Proceedings of the 32nd Annual Conference on Computer Security Applications*, 2016, pp. 520–532.
- [15] M. Kumar, M. K. Jindal, and M. Kumar, "A novel attack on monochrome and greyscale devanagari captchas," *ACM Transactions on Asian and Low-Resource Language Information Processing*, vol. 20, no. 4, pp. 1–30, 2021.
- [16] —, "An efficient technique for breaking of coloured hindi captcha," *Soft Computing*, vol. 27, no. 16, pp. 11 661–11 686, 2023.
- [17] T.-C. Yang, I. F. Ince, and Y. B. Salman, "A korean captcha study: defeating ocrs in a new captcha context by using korean syllables," *International Journal of Contents*, vol. 5, no. 3, pp. 50–56, 2009.
- [18] C. Shi, S. Ji, Q. Liu, C. Liu, Y. Chen, Y. He, Z. Liu, R. Beyah, and T. Wang, "Text captcha is dead? a large scale deployment and empirical study," in *Proceedings of the 27th ACM SIGSAC Conference on Computer and Communications Security*, 2020, pp. 1391–1406.
- [19] M. Kumar and M. K. Jindal, "Benchmarks for designing a secure devanagari captcha," *SN Computer Science*, vol. 2, no. 1, p. 45, 2021.
- [20] M. Kumar, M. K. Jindal, and M. Kumar, "Design of innovative captcha for hindi language," *Neural Computing and Applications*, vol. 34, no. 6, pp. 4957–4992, 2022.
- [21] D. Lin, F. Lin, Y. Lv, F. Cai, and D. Cao, "Chinese character captcha recognition and performance estimation via deep neural network," *Neurocomputing*, vol. 288, pp. 11–19, 2018.
- [22] Z. Cheng, H. Gao, Z. Liu, H. Wu, Y. Zi, and G. Pei, "Image-based captchas based on neural style transfer," *IET Information Security*, vol. 13, no. 6, pp. 519–529, 2019.
- [23] J. Yan and A. S. El Ahmad, "Usability of captchas or usability issues in captcha design," in *Proceedings of the 4th Symposium on Usable Privacy and Security*, 2008, pp. 44–52.
- [24] Y. Gao, H. Gao, S. Luo, Y. Zi, S. Zhang, W. Mao, P. Wang, Y. Shen, and J. Yan, "Research on the security of visual reasoning captcha," in *Proceedings of the 30th USENIX Security Symposium*, 2021, pp. 3291–3308.
- [25] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," in *Proceedings of the 3rd International Conference on Learning Representations*, 2015, pp. 1–11.
- [26] C. Xie, Z. Zhang, Y. Zhou, S. Bai, J. Wang, Z. Ren, and A. L. Yuille, "Improving transferability of adversarial examples with input diversity," in *Proceedings of the 28th IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019, pp. 2730–2739.
- [27] Y. Fu, G. Sun, H. Yang, J. Huang, and H. Wang, "Fighting attacks on large character set captchas using transferable adversarial examples," in *Proceedings of the 33rd International Joint Conference on Neural Networks*. IEEE, 2023, pp. 1–10.
- [28] J. Lin, C. Song, K. He, L. Wang, and J. E. Hopcroft, "Nesterov accelerated gradient and scale invariance for adversarial attacks," in *Proceedings of the 8th International Conference on Learning Representations*, 2020, pp. 1–12.
- [29] X. Wang and K. He, "Enhancing the transferability of adversarial attacks through variance tuning," in *Proceedings of the 30th IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021, pp. 1924–1933.
- [30] Y. Dong, T. Pang, H. Su, and J. Zhu, "Evading defenses to transferable adversarial examples by translation-invariant attacks," in *Proceedings*

- of the 28th IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2019, pp. 4312–4321.
- [31] Y. Xu, G. Reynaga, S. Chiasson, J.-M. Frahm, F. Monrose, and P. C. Van Oorschot, “Security analysis and related usability of motion-based captchas: Decoding codewords in motion,” *IEEE Transactions on Dependable and Secure Computing*, vol. 11, no. 5, pp. 480–493, 2013.
- [32] P. Wang, H. Gao, X. Guo, C. Xiao, F. Qi, and Z. Yan, “An experimental investigation of text-based captcha attacks and their robustness,” *ACM Computing Surveys*, vol. 55, no. 9, pp. 1–38, 2023.
- [33] H. Gao, M. Tang, Y. Liu, P. Zhang, and X. Liu, “Research on the security of microsoft’s two-layer captcha,” *IEEE Transactions on Information Forensics and Security*, vol. 12, no. 7, pp. 1671–1685, 2017.
- [34] M. Osadchy, J. Hernandez-Castro, S. Gibson, O. Dunkelman, and D. Pérez-Cabo, “No bot expects the deepcaptcha! introducing immutable adversarial examples, with applications to captcha generation,” *IEEE Transactions on Information Forensics and Security*, vol. 12, no. 11, pp. 2640–2653, 2017.
- [35] C. Shi, X. Xu, S. Ji, K. Bu, J. Chen, R. Beyah, and T. Wang, “Adversarial captchas,” *IEEE Transactions on Cybernetics*, vol. 52, no. 7, pp. 6095–6108, 2021.
- [36] R. Shao, Z. Shi, J. Yi, P.-Y. Chen, and C.-J. Hsieh, “Robust text captchas using adversarial examples,” in *Proceedings of the 10th IEEE International Conference on Big Data*, 2022, pp. 1495–1504.
- [37] J. Zhang, J. Sang, K. Xu, S. Wu, X. Zhao, Y. Sun, Y. Hu, and J. Yu, “Robust captchas towards malicious ocr,” *IEEE Transactions on Multimedia*, vol. 23, pp. 2575–2587, 2020.
- [38] I. Hossen and A. Hei, “aecaptcha: The design and implementation of audio adversarial captcha,” in *Proceedings of the 7th IEEE European Symposium on Security and Privacy*, 2022, pp. 430–447.
- [39] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, “Towards deep learning models resistant to adversarial attacks,” in *Proceedings of the 6th International Conference on Learning Representations*, 2018, pp. 1–23.
- [40] C. Donahue, J. McAuley, and M. Puckette, “Adversarial audio synthesis,” in *Proceedings of the 7th International Conference on Learning Representations*, 2019, pp. 1–16.
- [41] Y. Zhang, H. Gao, G. Pei, S. Kang, and X. Zhou, “Effect of adversarial examples on the robustness of captcha,” in *Proceedings of the 10th International Conference on Cyber-Enabled Distributed Computing and Knowledge Discovery*, 2018, pp. 1–109.
- [42] P. Wang, H. Gao, C. Xiao, X. Guo, Y. Gao, and Y. Zi, “Extended research on the security of visual reasoning captcha,” *IEEE Transactions on Dependable and Secure Computing*, vol. 20, no. 6, pp. 4976–4992, 2023.
- [43] M. Andriushchenko and N. Flammarion, “Understanding and improving fast adversarial training,” *Proceedings of the 34th Advances in Neural Information Processing Systems*, vol. 33, pp. 16 048–16 059, 2020.
- [44] C. Szegedy, W. Zaremba, I. Sutskever, J. Bruna, D. Erhan, I. J. Goodfellow, and R. Fergus, “Intriguing properties of neural networks,” in *2nd International Conference on Learning Representations*, 2014.
- [45] Y. Dong, F. Liao, T. Pang, H. Su, J. Zhu, X. Hu, and J. Li, “Boosting adversarial attacks with momentum,” in *Proceedings of the 27th IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2018, pp. 9185–9193.
- [46] W. Wu, Y. Su, X. Chen, S. Zhao, I. King, M. R. Lyu, and Y.-W. Tai, “Boosting the transferability of adversarial samples via attention,” in *Proceedings of the 29th IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020, pp. 1161–1170.
- [47] C. Li, W. Yao, H. Wang, and T. Jiang, “Adaptive momentum variance for attention-guided sparse adversarial attacks,” *Pattern Recognition*, vol. 133, p. 108979, 2023.
- [48] A. Kurakin, I. J. Goodfellow, and S. Bengio, “Adversarial examples in the physical world,” in *Proceedings of the 6th International Conference on Learning Representations*, 2018, pp. 1–14.
- [49] D. Wang, C. Li, S. Wen, Q.-L. Han, S. Nepal, X. Zhang, and Y. Xiang, “Daedalus: Breaking nonmaximum suppression in object detection via adversarial examples,” *IEEE Transactions on Cybernetics*, vol. 52, no. 8, pp. 7427–7440, 2021.
- [50] C. Xie, J. Wang, Z. Zhang, Y. Zhou, L. Xie, and A. Yuille, “Adversarial examples for semantic segmentation and object detection,” in *Proceedings of the 16th IEEE International Conference on Computer Vision*, 2017, pp. 1369–1378.
- [51] X. Wei, S. Liang, N. Chen, and X. Cao, “Transferable adversarial attacks for image and video object detection,” in *Proceedings of the 28th International Joint Conference on Artificial Intelligence*, 7 2019, pp. 954–960.
- [52] S.-T. Chen, C. Cornelius, J. Martin, and D. H. Chau, “Shapeshifter: Robust physical adversarial attack on faster r-cnn object detector,” in *Proceedings of the 30th European Conference on Machine Learning*, 2019, pp. 52–68.
- [53] Y. Zhao, H. Zhu, R. Liang, Q. Shen, S. Zhang, and K. Chen, “Seeing isn’t believing: Towards more robust adversarial attack against real world object detectors,” in *Proceedings of the 26th ACM SIGSAC Conference on Computer and Communications Security*, 2019, pp. 1989–2004.
- [54] S. Thys, W. Van Ranst, and T. Goedemé, “Fooling automated surveillance cameras: adversarial patches to attack person detection,” in *Proceedings of the 28th IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*, 2019.
- [55] N. Papernot, P. McDaniel, S. Jha, M. Fredrikson, Z. B. Celik, and A. Swami, “The limitations of deep learning in adversarial settings,” in *Proceedings of the IEEE 1st European Symposium on Security and Privacy*, 2016, pp. 372–387.
- [56] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, “Grad-cam: Visual explanations from deep networks via gradient-based localization,” in *Proceedings of the 16th IEEE International Conference on Computer Vision*, 2017, pp. 618–626.
- [57] B. Zhou, A. Khosla, A. Lapiedra, A. Oliva, and A. Torralba, “Learning deep features for discriminative localization,” in *Proceedings of the 25th IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2016, pp. 2921–2929.
- [58] Y. Xiong, J. Lin, M. Zhang, J. E. Hopcroft, and K. He, “Stochastic variance reduced ensemble adversarial attack for boosting the adversarial transferability,” in *Proceedings of the 31st IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 14 983–14 992.
- [59] C. Szegedy, S. Ioffe, V. Vanhoucke, and A. A. Alemi, “Inception-v4, inception-resnet and the impact of residual connections on learning,” in *Proceedings of the 31st Annual AAAI Conference on Artificial Intelligence*, 2017.
- [60] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, “Rethinking the inception architecture for computer vision,” in *Proceedings of the 25th IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2016, pp. 2818–2826.
- [61] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proceedings of the 25th IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2016, pp. 770–778.
- [62] K. Simonyan and A. Zisserman, “Very deep convolutional networks for large-scale image recognition,” in *Proceedings of the 3rd International Conference on Learning Representations*, 2015, pp. 1–14.
- [63] S. Ren, K. He, R. Girshick, and J. Sun, “Faster r-cnn: Towards real-time object detection with region proposal networks,” in *Proceedings of the 29th Advances in Neural Information Processing Systems*, 2015, pp. 91–99.
- [64] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. C. Berg, “Ssd: Single shot multibox detector,” in *Proceedings of the 14th European Conference on Computer Vision*, 2016, pp. 21–37.
- [65] G. Jocher et al., “Ultralytics/yolov5: v6.2,” 2022. Available online: <https://doi.org/10.5281/zenodo.3908559> (accessed on 26 August 2022). [Online]. Available: <https://github.com/ultralytics/yolov5>
- [66] S.-M. Moosavi-Dezfooli, A. Fawzi, and P. Frossard, “Deepfool: a simple and accurate method to fool deep neural networks,” in *Proceedings of the 25th IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2016, pp. 2574–2582.
- [67] A. Kurakin, I. Goodfellow, and S. Bengio, “Adversarial machine learning at scale,” in *Proceedings of the 5th International Conference on Learning Representations*, 2017, pp. 1–17.
- [68] H. Gao, J. Yan, F. Cao, Z. Zhang, L. Lei, M. Tang, P. Zhang, X. Zhou, X. Wang, J. Li et al., “A simple generic attack on text captchas,” in *Proceedings of the 23rd Annual Network and Distributed System Security Symposium*, 2016.
- [69] T. B. Brown, D. Mané, A. Roy, M. Abadi, and J. Gilmer, “Adversarial patch,” in *Proceedings of the 31st Advances in Neural Information Processing Systems*, 2017.
- [70] H. Huang, Z. Chen, H. Chen, Y. Wang, and K. Zhang, “T-sea: Transfer-based self-ensemble attack on object detection,” in *Proceedings of the 32nd IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 20 514–20 523.
- [71] X. Teoh, Y. Lin, S. Li, R. Liu, A. Sollomoni, Y. Harel, and J. S. Dong, “Are captchas still bot-hard? generalized visual captcha solving with agentic vision language model,” in *Proceedings of the 34th USENIX Security Symposium*, 2025, pp. 3747–3766.
- [72] Z. Wu, Y. Xue, X. Wei, and Y. Song, “Mca-bench: A multimodal benchmark for evaluating captcha robustness against vlm-based attacks,” *arXiv preprint arXiv:2506.05982*, 2025.

- [73] Z. Ding, G. Deng, Y. Liu, J. Ding, J. Chen, Y. Sui, and Y. Li, "Illusioncaptcha: A captcha based on visual illusion," in *Proceedings of the 34th ACM Web Conference*, 2025, pp. 3683–3691.
- [74] B. Miao, C. Li, Y. Zhu, W. Sun, Z. Wang, X. Wang, and C. Xie, "Advlogo: Adversarial patch attack against object detectors based on diffusion models," *arXiv preprint arXiv:2409.07002*, 2024.
- [75] E.-C. Chen, P.-Y. Chen, I. Chung, C.-R. Lee *et al.*, "Overload: Latency attacks on object detection for edge devices," in *Proceedings of the 33rd IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 24 716–24 725.
- [76] Z. Yang, J. Tang, Z. Li, P. Wang, J. Wan, H. Zhong, X. Liu, M. Yang, P. Wang, S. Bai *et al.*, "Cc-ocr: A comprehensive and challenging ocr benchmark for evaluating large multimodal models in literacy," in *Proceedings of the 34th IEEE/CVF International Conference on Computer Vision*, 2025, pp. 21 744–21 754.



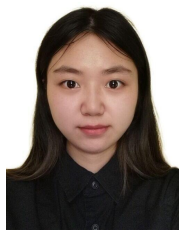
Ruimei Zhang received the Ph.D. degree in applied mathematics from the School of Mathematical Sciences, University of Electronic Science and Technology of China, Chengdu, China, in 2019. In 2020, she joined the School of Cyber Science and Engineering, Sichuan University, Chengdu, where she is currently an Associate Researcher. From 2017 to 2018, she was a Visiting Scholar with the Department of Applied Mathematics, University of Waterloo, Waterloo, ON, Canada. From 2018 to 2019, she was a Visiting Scholar with the Department of Electrical Engineering, Yeungnam University, Gyeongsan, South Korea. Her current research interests include neural networks, complex networks, multiagent systems, cyber-physical systems, and networked secure control.



Guoheng Sun is currently pursuing the B. E. degree in the School of Cyber Science and Engineering, Sichuan University, Chengdu, China. He is expected to graduate in June 2024. His research interests include trustworthy AI and efficient AI computing.



Yucheng Fu is currently pursuing the Bachelor of Engineering in the School of Cyber Science And Engineering at Sichuan University. He is expected to graduate in June 2024. His research interests include CAPTCHA and deep learning techniques.



Han Yang is currently pursuing the Bachelor of Engineering in the School of Cyber Science and Engineering at Sichuan University. She is expected to graduate in June 2024. Her research interests include trustworthy and reliable AI.



Haizhou Wang (Member, IEEE) received the B.E. and Ph.D. degrees from the College of Computer Science, Sichuan University, Chengdu, China, in 2008 and 2014, respectively. From 2013 to 2014, he visited the Department of Electrical and Computer Engineering, University of Toronto, Toronto, ON, Canada, under the supervision of Prof. J. Lieberher. He has been an Associate Professor with the School of Cyber Science and Engineering, Sichuan University, since 2018. His research interests include information security, network measurement, social network analysis, and opensource intelligence analysis.



Juntian Huang is a current undergraduate student pursuing the Bachelor of Engineering in the School of Cyber Science and Engineering at Sichuan University. He is expected to graduate in June 2025. His research interests include deep learning and adversarial examples.